

第 2 章 Python 基础

在正式开始学习如何用代码构建智能系统之前，我们首先需要选择一门合适的编程语言。如果说编程语言是人类与计算机沟通的桥梁，那么 Python 无疑是最流行、最接近人类自然语言的一座。

2.1 初识 Python

本节将作为你 Python 之旅的起点，回答三个基本问题：

- (1) Python 究竟是什么？
- (2) 为什么它成为了人工智能时代的首选语言？
- (3) 它的代码看起来是什么样子的？

让我们通过这三个问题，一起揭开 Python 的神秘面纱。

2.1.1 Python：一门“胶水语言”

Python 由荷兰数学与计算机科学研究中心的吉多·范罗苏姆 (Guido van Rossum) 设计并于 1991 年发布。与 C++ 或 Java 等语言相比，Python 的设计哲学强调代码的可读性和简洁性。这种哲学集中体现在 Python 的“彩蛋”——Python 之禅 (The Zen of Python) 中。

1. Python 的核心特性：

- (1) 易于学习：语法清晰，代码可读性高，接近英语。非常适合作为第一门编程语言。
- (2) 开源免费：任何人都可以使用和贡献。
- (3) 跨平台：一份代码，在 Windows, macOS, Linux 上都能运行。
- (4) 解释型语言：代码逐行执行，无需编译，调试和开发速度快。（可以与 C++ 等编译型语言做个简单对比，便于有基础的学生理解）。
- (5) 强大的“胶水”特性：能轻松地将其他语言（如 C/C++）编写的模块连接在一起，这也是它在科学计算领域大放异彩的原因之一。

2.1.2 Python 之用

Python 是一门通用的高级编程语言，其清晰的语法和强大的库生态系统使其在众多领域得到广泛应用，并吸引了不同背景的学习者。

1. 主要应用领域

(1) 人工智能与数据科学：这是 Python 最核心的应用领域。它被广泛用于机器学习、深度学习、数据分析、数据可视化和科学计算。主流的 AI 框架和数据处理库，如 PyTorch、TensorFlow、Scikit-learn、Pandas 和 NumPy，都以 Python 为主要接口。

(2) Web 开发：Python 可用于构建服务器端应用程序（后端）。通过 Django、Flask 等 Web 框架，开发者可以高效地构建从简单 API 到大型复杂网站的各类 Web 服务。

(3) 自动化与脚本：Python 可用于编写自动化脚本，以执行重复性任务，从而提高工作效率。应用场景包括系统管理、文件批量处理、自动化测试和网络数据采集（爬虫）。

(4) 其他领域：Python 的应用还延伸至科学与工程计算、桌面图形用户界面(GUI)应用、游戏开发、物联网(IoT)等多个方面。

2. 适合的学习人群

(1) 编程初学者：因其语法简单、接近自然语言，Python 通常被推荐为第一门编程语言，有助于初学者理解核心编程概念并快速获得实践成果。

(2) 科研人员与高校学生：来自不同学科（如物理、生物、金融、社会科学等）的研究者和学生，可以利用 Python 进行数据处理、数值模拟、统计分析和结果可视化。

(3) 软件开发者：对于已有其他编程语言（如 C++/Java/MATLAB）经验的开发者，Python 可作为一门高效的辅助语言，用于快速原型开发、工具链构建或项目粘合。

(4) 非程序员专业人士：任何希望通过自动化来优化工作流程的职场人士，都可以通过学习 Python 脚本编程来提升个人生产力。

现在，你已经对 Python 有了一个宏观的认识。它是一门强大、优雅且应用广泛的语言，正等待着你去探索。在下一节中，我们将亲自动手，在自己的电脑上搭建起 Python 开发环境，正式开启我们的编程之旅！

2.2 Python 环境配置

“工欲善其事，必先利其器。” 在我们开始学习编写 Python 代码之前，首要任务是在自己的计算机上搭建一个稳定、高效的开发环境。这个过程就像一位画家准备画板和颜

料，或者一位厨师准备案板和厨具。一个好的环境能让我们事半功倍，避免许多不必要的麻烦。

本节我们将介绍当前最主流的 Python 环境配置方案，并带你一步步完成安装。我们还会探讨虚拟环境、WSL2、Docker 等现代化开发工具的概念，它们将是你未来进行严肃项目开发时的得力助手。

2.2.1 安装方式概览（官方、Anaconda）

安装 Python 主要有两种途径：直接从 Python 官网下载安装，或是通过 Anaconda 这个专为科学计算设计的发行版来安装。

（1）官方 Python (python.org):

优点：环境纯净，只包含 Python 标准库，体积小。

缺点：需要手动安装额外的科学计算库，且包管理和环境隔离需要借助 pip 和 venv 等多个工具，对新手不够友好。

（2）Anaconda:

优点：一站式解决方案。它不仅包含了 Python 本体，还预装了上百个常用的科学计算库（如 NumPy, Pandas, Matplotlib 等）。更重要的是，它自带了强大的包管理器和环境管理器 conda，管理项目依赖非常方便。

缺点：安装包体积较大（通常几百 MB）。

其实，还有一种途径是通过 **Miniconda** 安装，它是 Anaconda 的一种精简版，二者唯一的区别在于初始包含的库的大小，Miniconda 仅包含几个必要的依赖包，因此环境相对干净、轻量。如果你有一定的经验并且希望对环境有完全的控制，可以尝试通过 Miniconda 安装，其与 Anaconda 的使用方式一摸一样。

教学建议：

对于初学者以及本书的目标读者而言，我们强烈推荐使用 Anaconda。它可以为你省去大量繁琐的配置工作，让你能从一开始就专注于 Python 编程和数据科学实践本身。本教材后续内容也将默认你使用的是基于 Anaconda 的环境。如果对官方 Python 感兴趣，可以自行在网上学习。

1. Anaconda 安装步骤（以 Windows 10/11 为例）

(1) 下载安装程序:

访问 Anaconda 官方下载页面: <https://www.anaconda.com/download/success>

选择你对应的操作系统 (这里以 Windows 为例), 点击 Anaconda 对应的图形化安装包 (Graphical Installer) 下载即可。

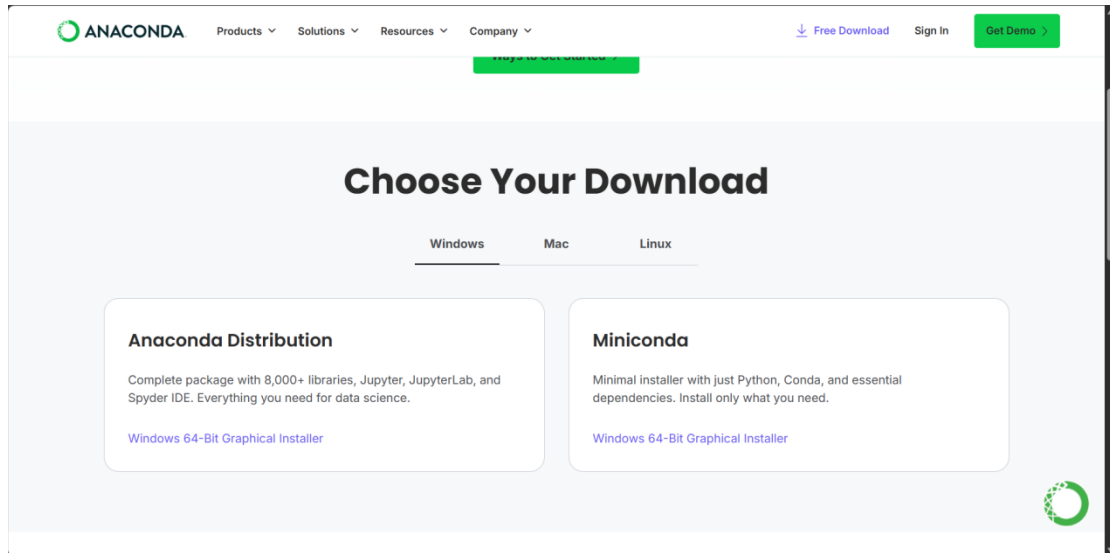
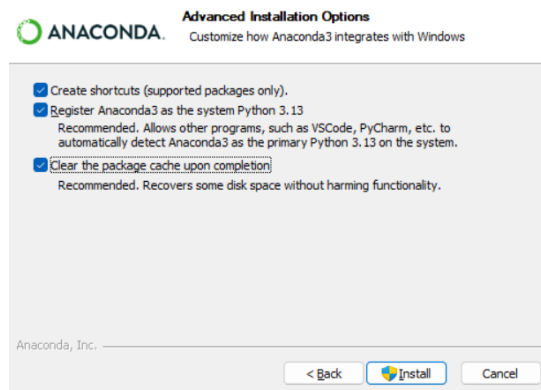


图 3.1 Anaconda 官网下载页面

(2) 运行安装程序:

双击下载好的 .exe 文件, 启动安装向导。

点击 Next -> 同意许可协议 I Agree -> 选择 All Users (为所有用户安装) 并点击 Next -> 选择 Anaconda 的安装位置 (这里建议有分盘的安装在 C 盘之外的**其他盘**, 注意**路径不能包含中文**, 然后**记住安装位置**, 后面会用) -> 把三个勾全部打上, 然后点击 Install, Anaconda 开始安装 -> 安装完成后点击 Next -> 两个勾全都取消, 不要框选, 点击 Finish, 完成安装



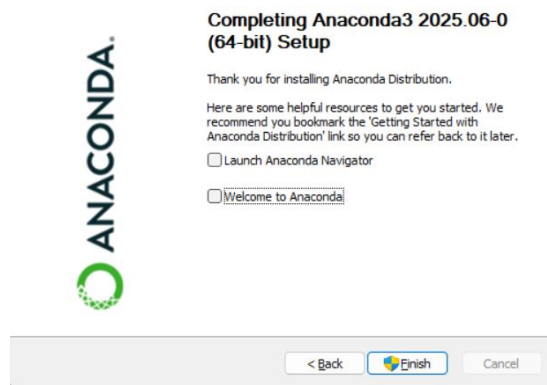


图 3.2 Anaconda 安装部分页面

(3) 配置环境变量：

按下 win 键，在搜索栏输入“环境变量”，在下方列表点击打开“编辑系统环境变量”。在弹出的窗口中点击打开“环境变量”，在“系统变量”一栏中，下拉找到 Path 一行并点击选中，点击“编辑”。在弹出的窗口中点击新建——添加以下路径。（注意：对应路径需改成读者在第 2 步安装 Anaconda 时的路径，这里以笔者的为例。）

D:\Anaconda3	# 主解释器路径
D:\Anaconda3\Scripts	# 核心命令行工具
D:\Anaconda3\Library\bin	# 通用二进制库

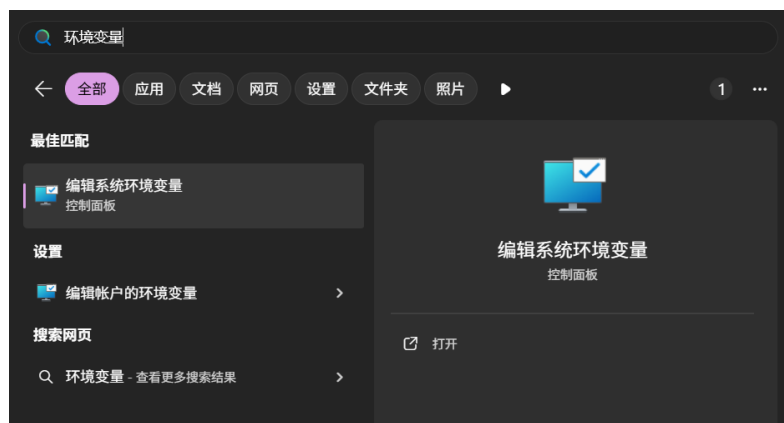


图 3.3 编辑系统环境变量打开页面



图 3.4 环境变量打开方式

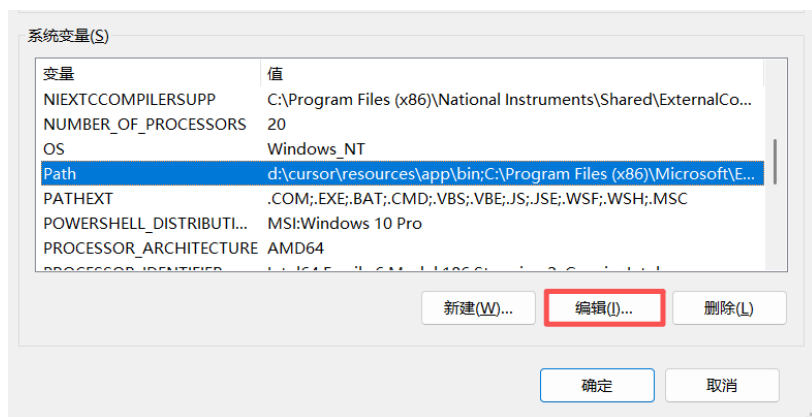


图 3.5 编辑环境变量

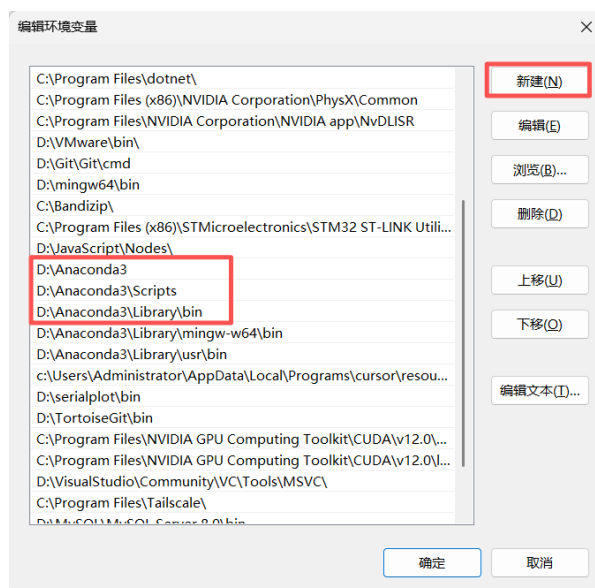


图 3.6 环境变量添加示例

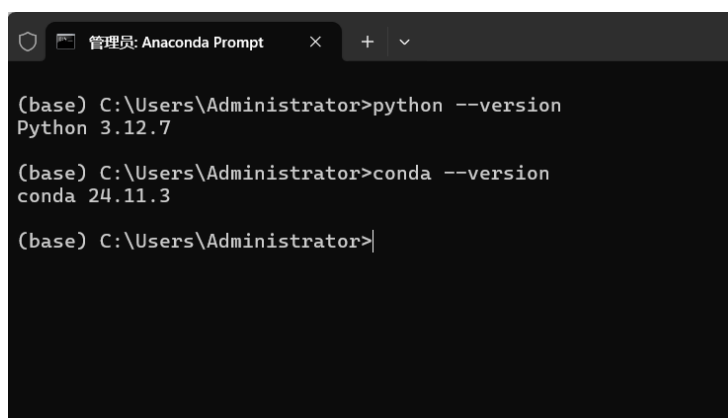
(4) 验证安装:

在 Windows 的开始菜单中, 输入并打开 Anaconda Prompt。这是一个专门为 Anaconda 配置好的命令行终端。

在打开的黑色窗口中, 输入以下命令并按回车: `python --version`

如果你看到类似 `Python 3.12.7` 这样的版本号信息, 恭喜你, Python 已经成功安装!

接着, 输入 `conda --version`, 应该能看到 `conda 24.11.3` 这样的版本信息。

A screenshot of the Anaconda Prompt terminal window. The window title is '管理员: Anaconda Prompt'. The terminal shows three commands and their outputs: 1. Command: `(base) C:\Users\Administrator>python --version`, Output: `Python 3.12.7`. 2. Command: `(base) C:\Users\Administrator>conda --version`, Output: `conda 24.11.3`. 3. Command: `(base) C:\Users\Administrator>` followed by a cursor, with no output shown yet.

```
(base) C:\Users\Administrator>python --version
Python 3.12.7

(base) C:\Users\Administrator>conda --version
conda 24.11.3

(base) C:\Users\Administrator>
```

图 3.7 在 Anaconda Prompt 中验证安装

然后打开命令行终端, 输入相同的命令, 应该能看到一样的输出, 说明环境变量已成功配置。

2.2.2 虚拟环境

在进行项目开发时, 我们经常会遇到一个问题: 项目 A 需要 LibraryX 的 1.0 版本, 而项目 B 却需要 LibraryX 的 2.0 版本。如果我们将所有库都安装在同一个全局环境中, 这两个项目就无法同时正常工作。

虚拟环境 (Virtual Environment) 就是为了解决这个问题而生的。它能为每个项目创建一个独立的、与外界隔离的 Python 环境。

conda 提供了非常简单易用的虚拟环境管理功能。

1. 使用 conda 管理虚拟环境

所有操作都在 Anaconda Prompt 中进行。

(1) 创建新环境:

我们来为一个名为 `conda_test` 的项目创建一个新的虚拟环境，并指定使用 Python 3.10 版本。

```
# conda create --name <环境名> python=<版本号>
```

```
conda create --name conda_test python=3.10
```

随后终端会提示你将要安装哪些包，输入 `y` 并回车确认，等待创建完毕。

```
C:\Users\Administrator>conda create --name conda_test python=3.10
Retrieving notices: done
Channels:
- defaults
Platform: win-64
Collecting package metadata (repodata.json): done
Solving environment: done

## Package Plan ##

environment location: D:\Anaconda3\envs\conda_test

added / updated specs:
- python=3.10

The following packages will be downloaded:

package                                     build                                     128 KB
ca-certificates-2025.11.4                  haa95532_0
expat-2.7.3                                h9214b88_0                               134 KB
libzlib-1.3.1                              h02ab6af_0                               64 KB
openssl-3.0.18                             h543e019_0                              6.8 MB
pip-25.3                                   pyhc872135_0                             1.1 MB
python-3.10.19                             h981015d_0                              15.3 MB
setuptools-80.9.0                         py310haa95532_0                          1.4 MB
sqlite-3.51.0                             hda9a48d_0                               917 KB
tk-8.6.15                                 hf199647_0                               3.5 MB
ucrt-10.0.22621.0                         haa95532_0                               620 KB
vc-14.3                                   h2df5915_10                             19 KB

ca-certificates      pkgs/main/win-64::ca-certificates-2025.11.4-haa95532_0
expat                pkgs/main/win-64::expat-2.7.3-h9214b88_0
libffi               pkgs/main/win-64::libffi-3.4.4-hd77b12b_1
libzlib              pkgs/main/win-64::libzlib-1.3.1-h02ab6af_0
openssl              pkgs/main/win-64::openssl-3.0.18-h543e019_0
pip                  pkgs/main/noarch::pip-25.3-pyhc872135_0
python               pkgs/main/win-64::python-3.10.19-h981015d_0
setuptools            pkgs/main/win-64::setuptools-80.9.0-py310haa95532_0
sqlite               pkgs/main/win-64::sqlite-3.51.0-hda9a48d_0
tk                   pkgs/main/win-64::tk-8.6.15-hf199647_0
tzdata               pkgs/main/noarch::tzdata-2025b-h04d1e01_0
ucrt                  pkgs/main/win-64::ucrt-10.0.22621.0-haa95532_0
vc                    pkgs/main/win-64::vc-14.3-h2df5915_10
vc14_runtime          pkgs/main/win-64::vc14_runtime-14.44.35208-h927774_10
vs2015_runtime         pkgs/main/win-64::vs2015_runtime-14.44.35208-ha6b5a95_10
wheel                 pkgs/main/win-64::wheel-0.45.1-py310haa95532_0
xz                    pkgs/main/win-64::xz-5.6.4-h4754444_1
zlib                  pkgs/main/win-64::zlib-1.3.1-h02ab6af_0

Proceed ([y]/n)? y

Downloading and Extracting Packages:
python-3.10.19      | 15.3 MB | ##### 100%
openssl-3.0.18     | 6.8 MB | ##### 100%
tk-8.6.15          | 3.5 MB | ##### 100%
setuptools-80.9.0  | 1.4 MB | ##### 100%
pip-25.3           | 1.1 MB | ##### 100%
sqlite-3.51.0      | 917 KB | ##### 100%
vc14_runtime-14.44.3 | 825 KB | ##### 100%
ucrt-10.0.22621.0  | 620 KB | ##### 100%

tk-8.6.15          | hf199647_0 | 3.5 MB
ucrt-10.0.22621.0  | haa95532_0 | 620 KB
vc-14.3            | h2df5915_10 | 19 KB
vc14_runtime-14.44.35208 | h4927774_10 | 825 KB
vs2015_runtime-14.44.35208 | ha6b5a95_10 | 19 KB
zlib-1.3.1         | h02ab6af_0 | 113 KB
Total: 31.0 MB

The following NEW packages will be INSTALLED:
bzip2                pkgs/main/win-64::bzip2-1.0.8-h2b8ff1b_6
ca-certificates       pkgs/main/win-64::ca-certificates-2025.11.4-haa95532_0
expat                 pkgs/main/win-64::expat-2.7.3-h9214b88_0
libffi                pkgs/main/win-64::libffi-3.4.4-hd77b12b_1
libzlib               pkgs/main/win-64::libzlib-1.3.1-h02ab6af_0
openssl               pkgs/main/win-64::openssl-3.0.18-h543e019_0
pip                   pkgs/main/noarch::pip-25.3-pyhc872135_0
done
#
# To activate this environment, use
#
#   $ conda activate conda_test
#
# To deactivate an active environment, use
#
#   $ conda deactivate
#
C:\Users\Administrator>
```

图 3.8 在 Anaconda Prompt 中创建虚拟环境

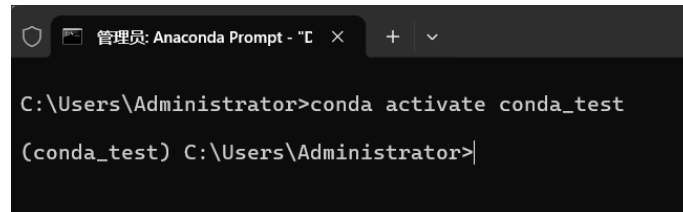
(2) 激活环境:

创建好后，需要“进入”这个环境才能使用它。


```
# conda activate <环境名>
```

```
conda activate conda_test
```

激活后，你会发现命令行提示符前面多了 (conda_test) 的字样，这表示你当前正处于这个虚拟环境中。



```
管理员: Anaconda Prompt - "C" x + v
C:\Users\Administrator>conda activate conda_test
(conda_test) C:\Users\Administrator>
```

图 3.9 激活 conda 虚拟环境

(3) 在环境中安装库：

激活环境后，我们可以使用 `conda install` 或 `pip install` 来安装库。这些库将只被安装到当前环境中。`pip` 是 Python 官方的包管理器，通常用于安装 conda 源中没有的库。

一般优先使用 conda 安装

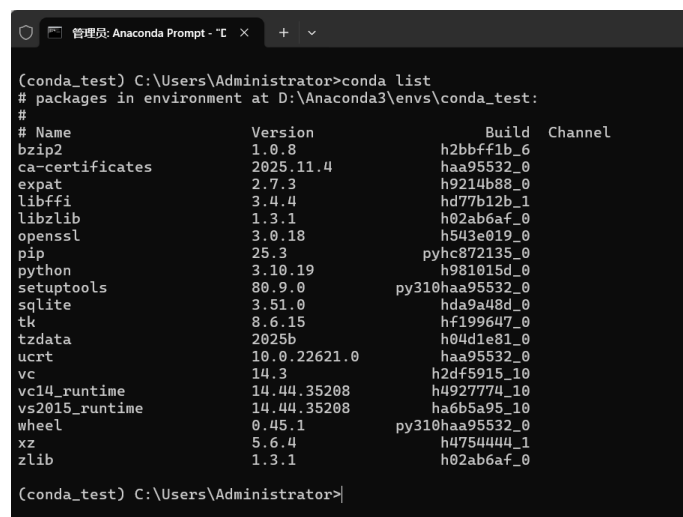
```
# conda install <库名>
```

如果 conda 源没有，再用 pip

```
# pip install <库名>
```

(4) 查看已安装的库：

```
# conda list
```



```
(conda_test) C:\Users\Administrator>conda list
# packages in environment at D:\Anaconda3\envs\conda_test:
#
# Name                  Version           Build    Channel
bzip2                   1.0.8             h2bfff1b_6
ca-certificates         2025.11.4         haa95532_0
expat                   2.7.3             h9214b88_0
libffi                  3.4.4             hd77b12b_1
libzlib                 1.3.1             h02ab6af_0
openssl                 3.0.18            h543e019_0
pip                     25.3              pyhc872135_0
python                  3.10.19           h981015d_0
setuptools              80.9.0            py310haa95532_0
sqlite                  3.51.0            hda9a48d_0
tk                       8.6.15            hf199647_0
tzdata                  2025b             h04d1e81_0
ucrt                    10.0.22621.0     haa95532_0
vc                       14.3              h2df5915_10
vc14_runtime            14.44.35208       h4927774_10
vs2015_runtime          14.44.35208       ha6b5a95_10
wheel                   0.45.1            py310haa95532_0
xz                       5.6.4             h4754444_1
zlib                    1.3.1             h02ab6af_0

(conda_test) C:\Users\Administrator>
```

图 3.10 查看现处环境已安装的库

(5) 退出环境：

当你完成了在这个项目上的工作，可以退出当前环境，返回到基础 (base) 环境。

```
# conda deactivate
```

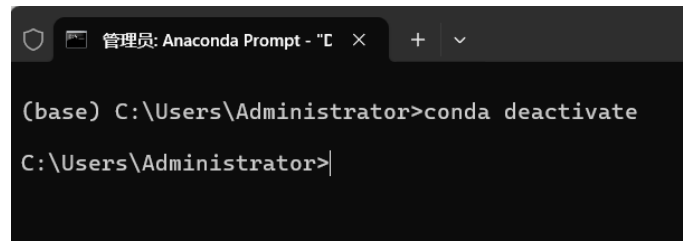


图 3.11 退出虚拟环境

你会发现提示符前面的 (conda_test) 消失了。

(6) 查看已有环境

你可以输入以下命令查看本机中存在的所有用 conda 创建的虚拟环境。

```
# conda env list
```

```
# 输出: <环境名> <环境所在路径>
```



图 3.12 查看已有的虚拟环境

(7) 删除虚拟环境

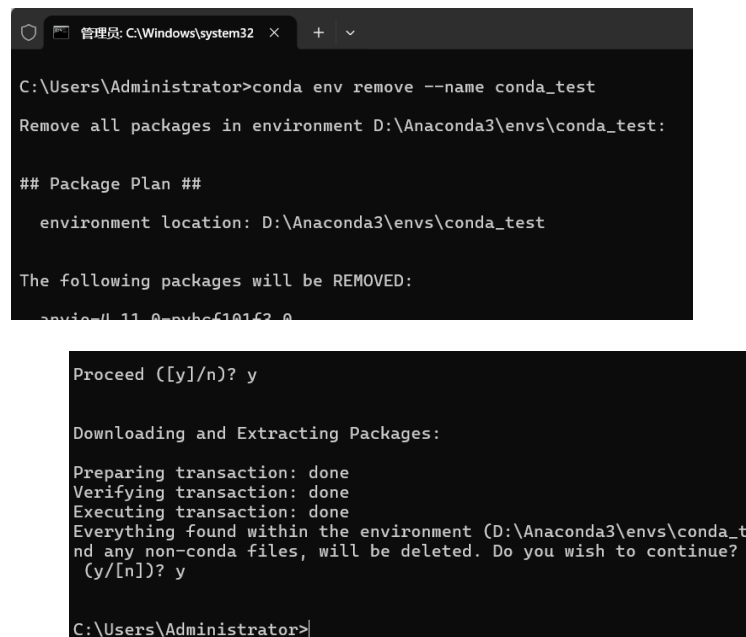
当一个项目完成或不再需要某个环境时，我们可以将其删除以释放硬盘空间。请注意，这是一个不可逆的操作，会永久删除该环境及其中的所有库。

在删除前，请务必先退出该环境。

```
# conda env remove --name <环境名>
```

```
conda env remove --name conda_test
```

执行命令后，conda 会列出该环境的路径并进行删除，过程中需输入 y 并回车确认。



```
C:\Users\Administrator>conda env remove --name conda_test
Remove all packages in environment D:\Anaconda3\envs\conda_test:

## Package Plan ##

  environment location: D:\Anaconda3\envs\conda_test

The following packages will be REMOVED:

python==11.0-pyhf510162_0

Proceed ([y]/n)? y

Downloading and Extracting Packages:

Preparing transaction: done
Verifying transaction: done
Executing transaction: done
Everything found within the environment (D:\Anaconda3\envs\conda_t
nd any non-conda files, will be deleted. Do you wish to continue?
(y/[n])? y

C:\Users\Administrator>
```

图 3.13 删除已有的虚拟环境

(8) 备份与迁移环境：共享你的工作空间

这是 conda 一个极其强大的功能。想象一下这些场景：

- 你需要和同学合作一个项目，如何保证他使用的库和版本与你完全一致？
- 你在本地电脑上完成了代码调试，现在需要部署到服务器上运行。
- 你准备重装电脑系统，如何快速恢复之前辛苦配置好的开发环境？

答案就是将环境导出为一个名为 `environment.yml` 的配置文件。这个文件就是一张“菜单”，精确记录了当前环境的名称、所有依赖库及其版本号。任何人拿到这张“菜单”，都能在自己的机器上一键复制出和你一模一样的环境。

a. 导出（备份）当前环境

首先，需要激活你想要导出的环境。然后执行导出命令，它会将配置信息输出并保存到目前目录下的 `environment.yml` 文件中。

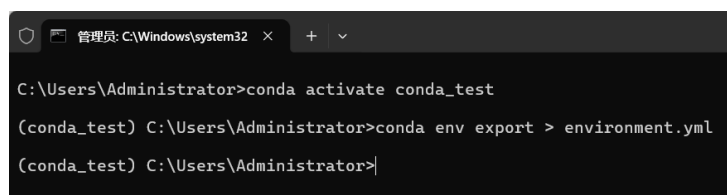
首先激活环境

```
# conda activate conda_test
```

然后导出配置

```
# conda env export > environment.yml
```

执行后，你的当前文件夹下会多出一个 `environment.yml` 文件。你可以用文本编辑器打开它，看看里面记录了哪些信息。



```
C:\Users\Administrator>conda activate conda_test
(conda_test) C:\Users\Administrator>conda env export > environment.yml
(conda_test) C:\Users\Administrator>
```

图 3.14 成功导出环境配置文件

b. 从文件创建（迁移）环境

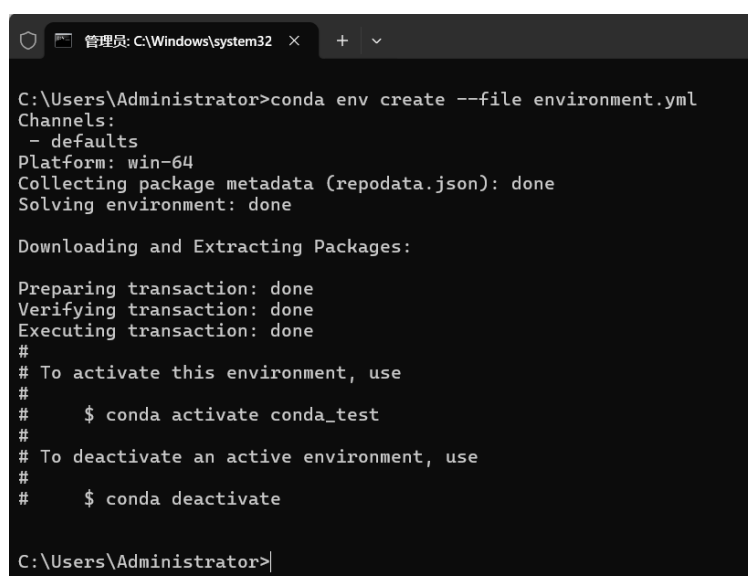
现在，你的同事（或者在另一台新电脑上的你）拿到了这个 `environment.yml` 文件。他只需一条命令，就可以完美复刻你的开发环境。

注意：在创建前，请确保当前没有任何已激活的环境（处于 `base` 环境下）。

`conda env create --file <文件名.yml>`

```
conda env create --file environment.yml
```

Conda 会读取 `environment.yml` 文件，自动创建一个名字和配置都与源环境相同的全新虚拟环境，并下载安装所有指定的库。这极大地保证了协作开发和项目部署的一致性。



```
C:\Users\Administrator>conda env create --file environment.yml
Channels:
 - defaults
Platform: win-64
Collecting package metadata (repodata.json): done
Solving environment: done

Downloading and Extracting Packages:

Preparing transaction: done
Verifying transaction: done
Executing transaction: done
#
# To activate this environment, use
#
#   $ conda activate conda_test
#
# To deactivate an active environment, use
#
#   $ conda deactivate
#
C:\Users\Administrator>
```

图 3.15 从配置文件一键恢复环境

Tips: 为每一个新项目创建一个专属的虚拟环境。这是一个专业开发者的基本素养，能保证项目的可复现性和环境的纯净性。

2.2.3 WSL2 与 Docker：更专业的开发与部署工具

对于初学者来说，掌握 Anaconda 已经足够应对大部分的 Python 开发。但如果你想了解更前沿、更专业的开发模式，WSL2 和 Docker 是你不可不知的两个概念。本节只做简要介绍，可自行前往互联网了解学习。

(1) WSL2 (Windows Subsystem for Linux 2)

是什么：它允许你在 Windows 系统内部，无缝地运行一个完整的、真实的 Linux 系统（如 Ubuntu）。

为什么 AI 开发需要它：许多 AI 工具、服务器软件和部署脚本都是基于 Linux 设计的。WSL2 为 Windows 用户提供了一个与生产环境几乎完全一致的开发环境，避免了许多跨平台兼容性问题。由于本书的第 2 章介绍了 Linux 基础，使用 WSL2 能让你学以致用。

快速上手：Windows 10/11 用户可以在 PowerShell 中以管理员身份运行 `wsl --install` 来一键安装。

(2) Docker

是什么：Docker 是一种“容器化”技术。你可以把它想象成一个软件的集装箱。它能将你的应用程序、其所有的依赖库、甚至操作系统环境，一起打包成一个标准的、轻量级的“镜像”(Image)。

解决了什么问题：解决了那个程序员最头疼的问题——“在我电脑上明明是好的！”。通过 Docker 镜像，你可以保证你的代码在任何安装了 Docker 的机器上（你的电脑、同学的电脑、云服务器），都能以一模一样的环境运行。这对于保证科研的可复现性至关重要。

快速上手：安装 Docker Desktop 后，在终端运行 `docker run hello-world`，快速体验一个容器的启动与退出。

2.2.4 Jupyter / IPython 的交互式优势

Anaconda 自带了一个强大的交互式编程工具——**Jupyter Notebook/Lab**。它特别适合数据分析、算法实验和教学演示。

IPython: Jupyter 的核心是一个增强版的 Python 解释器——IPython。它比原生的 Python shell 提供了更多功能，如代码自动补全、语法高亮、历史记录等。

Jupyter Notebook/Lab: 这是一个基于网页的交互式开发环境。你可以创建一种特殊的文件（.ipynb），在其中将代码、代码的运行结果、Markdown 文本、数学公式、图片等组合在一起，形成一份“可执行的实验报告”。

1. 启动 Jupyter Lab

- (1) 打开 Anaconda Prompt。
- (2) 激活你为项目创建的虚拟环境：`conda activate conda_test`
- (3) 在该环境中，安装 Jupyter Lab：`conda install -c conda-forge jupyterlab`
- (4) 安装完成后，输入命令启动：`# jupyter lab`

你的默认浏览器会自动打开一个新标签页，显示 Jupyter Lab 的界面。现在，你就可以在里面新建 Notebook，开始你的交互式编程之旅了！

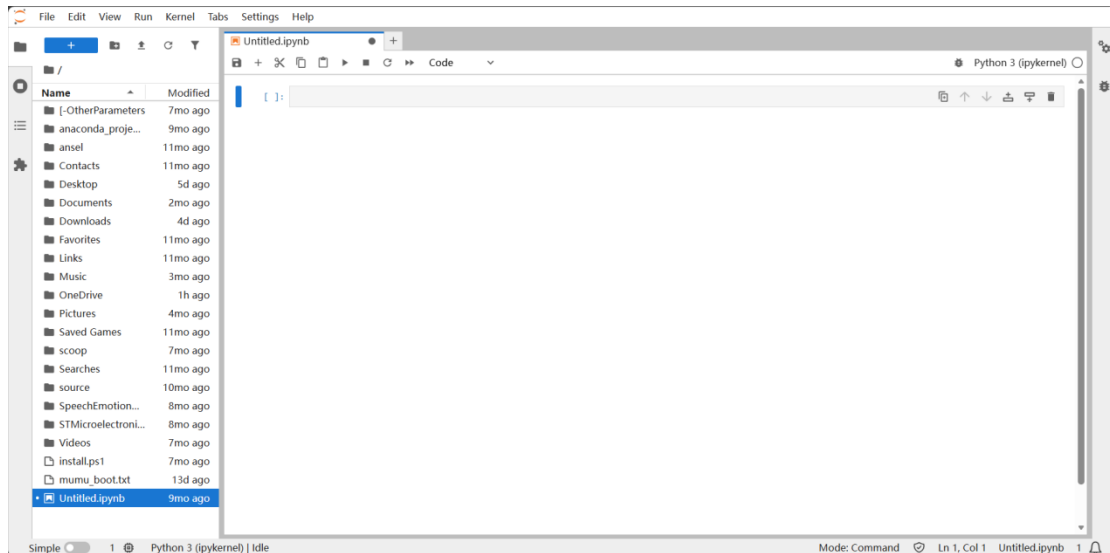


图 3.16 Jupyter Lab 界面

Jupyter 的核心优势在于其“单元格”(Cell) 执行模式。你可以将代码拆分成多个块，独立运行并立即查看结果。这个“编写一小块、运行、观察、再编写”的循环，极大地提高了数据探索和模型调试的效率。我们将在后续的章节中大量使用它。

2.3 IDE 介绍与 AI 辅助工具

在配置好 Python 环境后，我们需要一个强大的“代码编辑器”或“集成开发环境”(IDE, Integrated Development Environment) 来编写、调试和管理我们的代码。一个好的 IDE 就像一位智能副驾，能帮你发现错误、提示语法、导航代码，极大地提升开发效率。

本节我们将重点介绍目前最受欢迎的免费代码编辑器 Visual Studio Code (VS Code)，并探讨如何利用革命性的 AI 编程助手，让编写代码的体验进入下一个时代。

2.3.1 VS Code：现代化的代码编辑器

VS Code 是由微软开发的一款轻量级但功能极其强大的代码编辑器。它通过丰富的插件生态系统，几乎可以支持任何编程语言，并且对 Python 的支持尤为出色。

1. 安装与配置

(1) 下载：访问 VS Code 官网 <https://code.visualstudio.com/> 并下载对应你操作系统的版本。

(2) 安装：运行安装程序，建议在安装过程中勾选所有“其他”选项，特别是“将‘通过 Code 打开’操作添加到...”，这能让你在文件浏览器中右键文件夹或文件，直接用 VS Code 打开。

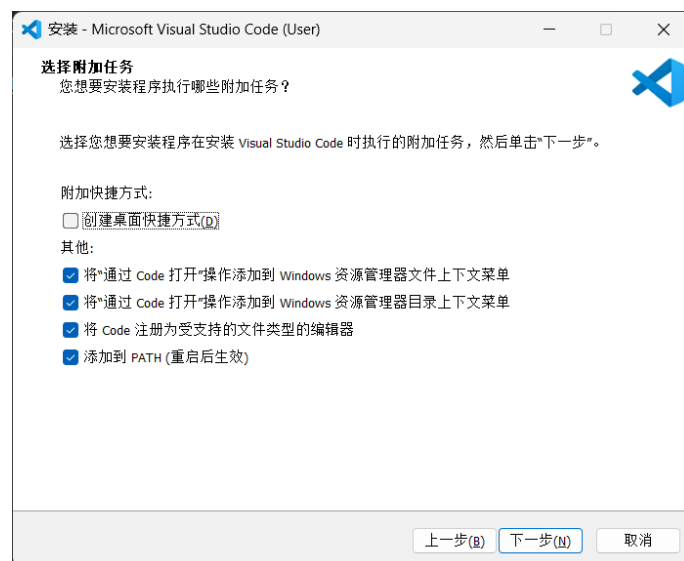


图 3.17 VS Code 安装界面

(3) 安装 Python 插件：

打开 VS Code，点击左侧边栏的“扩展”图标（四个方块的形状）。

在搜索框中输入 Python，选择由 Microsoft 发布的官方插件，点击 Install。这个插件是 VS Code 拥有强大 Python 功能的核心。

同样的方式，搜索并安装 Pylance 插件（通常会作为 Python 插件的依赖自动安装），它提供了顶级的代码补全和类型检查功能。

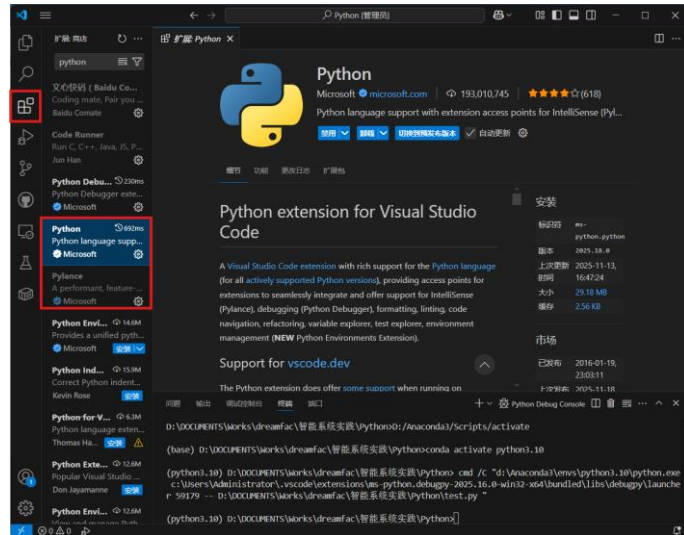


图 3.18 在 VS Code 中安装 Python 扩展

(4) 选择 Python 解释器 (关键步骤):

VS Code 需要知道使用哪个 Python 环境来运行和调试你的代码。

使用快捷键 **Ctrl+Shift+P** 打开命令面板，输入 **Python: Select Interpreter (Python: 选择解释器)**。

在弹出的列表中，VS Code 会自动检测你系统中已安装的 Python 环境，包括我们之前用 conda 创建的 conda_test 环境。选择它！

一旦选定，VS Code 窗口的右下角状态栏会显示你当前使用的 Python 版本和环境名。

2. VS Code 常用核心功能 (具体见 VS Code 官方文档)

假设我们有一个简单的 Python 文件 main.py:

```
import math

def calculate_circle_area(radius):
    """计算圆的面积"""
    if radius < 0:
        print("错误: 半径不能为负数")
        return None
    area = math.pi * (radius ** 2)
    return area
```



```
result = calculate_circle_area(10)

print(f'面积是: {result}')
```

(1) 代码导航 (Code Navigation):

按住 **Ctrl** 并单击函数名 `calculate_circle_area`，可以立即跳转到它的定义处。

在函数或变量上右键，选择“查找所有引用”，可以找到项目中所有使用到它的地方。

(2) 调试 (Debugging):

这是 IDE 最强大的功能之一。它允许你逐行运行代码，并检查每一步的变量状态。

设置断点：在代码行号的左侧单击，会出现一个红点，这就是“断点”。当程序运行到这里时，会暂停执行。我们在 `“area = ...”` 这一行设置一个断点。

启动调试：按 **F5** 键（或点击左侧的“运行和调试”图标后点击绿色三角按钮）。

程序会在断点处暂停，此时：

- 左侧的“变量”窗口会显示所有当前作用域内变量的值（如 `radius`）。
- 顶部的调试工具栏可以让你控制程序的执行：单步跳过 (**F10**)、单步进入 (**F11**)、继续 (**F5**) 等。

(3) 集成终端 (Integrated Terminal):

使用快捷键 ``Ctrl+``` (反引号键) 可以在 VS Code 内部打开一个终端。

这个终端会自动激活你为当前项目选择的 Python 解释器环境（例如 `conda_test`），你可以在这里直接运行 `python main.py` 或安装新的包 `pip install requests`。

3. 在 VS Code 中使用 Jupyter Notebook

除了编写传统的 `.py` 脚本文件，VS Code 还提供了对 Jupyter Notebook (`.ipynb` 文件) 的一流支持。这让你可以在同一个编辑器中，同时享受脚本开发的严谨性和 Notebook 探索的灵活性。

(1) 安装插件和依赖

为了让 VS Code 能够运行 Jupyter Notebook，你需要安装 Jupyter 扩展，并确保在你的 Python 环境（例如 `conda_test`）中安装了 `jupyter` 包。（详见 3.2.4 节）

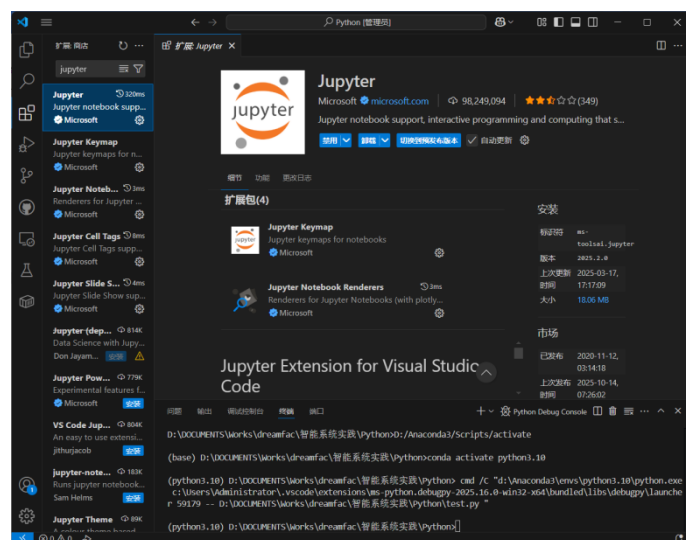


图 3.19 在 VS Code 中安装 Jupyter 扩展

(2) 创建并运行 Notebook

a. 新建 Notebook 文件：

在 VS Code 的文件浏览器中，点击“新建文件”图标。

将文件命名为“.ipynb”后缀。VS Code 会自动识别 .ipynb 后缀，并以 Notebook 的形式打开它。

b. 选择内核 (Kernel)：

“内核”就是执行 Notebook 代码的 Python 环境。

打开 .ipynb 文件后，点击右上角的“选择内核 (Select Kernel)”按钮。

在弹出的列表中，选择我们之前创建的 conda_test 环境。这确保了 Notebook 使用的是我们指定的、包含所有依赖库的独立环境。

c. 运行代码单元格：

点击单元格左侧的三角播放按钮，或使用快捷键 Ctrl+Enter 来运行它。

代码的输出，包括打印的文本和生成的图表，会直接显示在该单元格的下方。

通过将 Jupyter Notebook 集成到 VS Code 中，你获得了一个功能全面、体验统一的 Python 开发环境，足以应对从快速原型验证到复杂项目开发的各种需求。

2.3.2 AI 辅助编程：未来的编程范式

在我们熟练使用 IDE 提升效率的同时，一种革命性的新技术正在重塑编程的未来——那就是人工智能编程助手，其中最具代表性的有：GitHub Copilot、Cursor 等。它们

就像一个经验极其丰富的编程助手，能根据你的代码上下文和注释，实时为你提供代码建议。这些建议小到补全一个变量名，大到生成一整个函数或类。因此可以极大的提高我们的编程速度和编程质量。

尽管 AI 助手功能强大，但我们必须建立一个正确的认知：**AI 是辅助工具，而不是思考的替代品**。你需要去理解、审查、测试 AI 提供的代码，而不是盲目地复制粘贴。把它当作一个能极大加速你工作的助手，而不是一个可以替代你思考的“黑盒”。

总而言之，拥抱 AI 编程助手，并学会驾驭它，将使你在未来的学习和职业生涯中获得巨大的竞争优势。

通过此前的学习，你已经做好了 Python 所需的所有前期准备工作。接下来，我们将正式进入 Python 的核心语法世界，学习这门强大而简洁的语言！

2.4 Python 基础语法

现在你已经拥有了强大的代码编辑器和配置好的 Python 环境，是时候写下你的第一行真正的 Python 代码了。在本节中，我们将学习编程中最基础、最核心的几个概念，它们是构建一切复杂程序的起点。

2.4.1 输入与输出

程序通常需要从用户那里获取信息（输入），处理后展示结果（输出），这是最基本的人机交互。而在 Python 中这两种交互通常使用 `print()` 和 `input()` 来实现。

1. `print()` 函数

`print()` 是 Python 中最常用的内置函数之一，它的作用是将括号内的内容输出到屏幕（终端）上。

```
# 打印固定的字符串
print("Hello, world!")
```

```
# 打印变量的值
username = "Alice"
print(username)
```

（1）基础用法与分隔符 (sep)

`print()` 可以一次打印多个内容，默认使用空格分隔。你可以通过 `sep` 参数修改这个分隔符。

```
print("Python", "is", "fun")  
# -> Python is fun (默认空格)
```

```
print("Python", "is", "fun", sep="-")  
# -> Python-is-fun (用短横线连接)
```

```
print("2023", "10", "01", sep="/")  
# -> 2023/10/01
```

(2) 结束符 (`end`)

默认情况下，每次调用 `print()` 都会在末尾自动换行。如果你希望多次打印的内容连在同一行（例如打印进度条），可以修改 `end` 参数。

```
print("正在下载...", end="") # 不换行，结尾什么都不加  
print("100%")  
# -> 正在下载...100% (在同一行显示)
```

(3) 字符串格式化 (String Formatting)

这是本节的重点。在实际开发中，我们经常需要把变量嵌入到一段文字中，比如：“训练完成，准确率为 98.5%”。Python 提供了几种方式，这里仅介绍 `f-string` 和 `.format()` 方法。

(4) `f-string` (Python 3.6+)

在字符串的引号前加上字母 `f`，然后在字符串内部使用花括号 `{}` 包裹变量名。它直观、高效。

```
name = "Alice"  
score = 95.5
```

1. 直接嵌入变量

```
print(f"学生 {name} 的分数为 {score}")
```

2. 嵌入表达式 (直接在花括号里做运算)

```
print(f'明年 {name} 将会是 {20 + 1} 岁')
```

3. 控制数字格式 (AI 领域常用!)

:.2f 表示保留两位小数 (float)

```
acc = 0.87654321
```

```
print(f'模型准确率: {acc:.2f}') # -> 模型准确率: 0.88
```

```
print(f'模型准确率: {acc:.1%}') # -> 模型准确率: 87.7% (自动转百分比)
```

(5) .format()

这是 Python 3 早期的主流方式。你会在很多现存的代码库中看到它。

使用 {} 作为占位符, 后面跟 .format() 填充数据

```
print("学生 {} 的分数为 {}".format(name, score))
```

也可以指定顺序

```
print("{1} 的分数是 {0}".format(score, name))
```

print() 是你调试代码、查看程序运行状态的最基本、最有效的工具。

2. input() 函数

input() 函数会让程序暂停, 等待用户在终端中输入一些文字, 并按下回车。用户输入的内容将作为字符串返回。

提示用户输入名字, 并将输入的内容存入 name 变量

```
name = input("Please enter your name: ")
```

向用户打招呼

```
print(f'Hello, {name}!')
```

当你运行这段代码, 程序会先打印 Please enter your name: , 然后光标会停在那里闪烁, 等待你输入, 当你输入完毕, 按下回车, 程序会将你输入的内容返回并存入 name 变量中。

(1) 类型转换 (Type Casting)

需要注意的是，`input()` 函数返回的永远是**字符串**（详见 3.5.1 节数据类型）。如果你需要用户输入一个数字并进行计算，必须手动进行**类型转换**。

```
age_str = input("How old are you? ") # 假设用户输入 25

# age_str 的值是 "25" (一个字符串), 而不是 25 (一个整数)
# age_in_ten_years = age_str + 10 # 这行代码会报错! (字符串不能和整数相加)

# 正确的做法: 使用 int() 函数将字符串转换为整数
age_int = int(age_str)
age_in_ten_years = age_int + 10

print(f"In ten years, you will be {age_in_ten_years} years old.")
```

(2) 一次输入多个值 (Input Multiple Values)

有时候，我们希望用户在同一行输入多个数据，中间用空格隔开（例如输入坐标 10 20）。如果直接使用 `input()`，它会把 "10 20" 当作一个**完整的字符串**读取进来。

为了把它们分开，我们需要使用字符串的 `.split()` 方法。它就像一把剪刀，默认以“空格”为标记，将长字符串剪成多个短字符串。

a. 输入多个字符串

假设我们需要用户一次性输入姓和名。

```
# 提示用户在一行内输入，用空格隔开
# 例如输入: Alice Smith

full_name = input("Please enter your First Name and Last Name: ")

# 使用 split() 进行切割
# 如果输入是 "Alice Smith", split() 会把它变成列表 ['Alice', 'Smith']
# 然后我们把这两个值分别赋值给 first_name 和 last_name

first_name, last_name = full_name.split()

print(f"First Name: {first_name}")
print(f"Last Name: {last_name}")
```

b. 输入多个数字（进阶技巧）

如果输入的是数字（如 10 20），`.split()` 切割后得到的依然是字符串 `['10', '20']`。如果需要计算，我们需要结合 `map()` 函数来实现“批量转换”。

提示：这行代码在算法题和数据处理中非常常用，建议背下来。

```
# 提示用户输入矩形的长和宽，用空格隔开
# 例如输入：10 5

input_str = input("Please enter width and height (e.g., 10 5): ")

# 1. input_str.split() -> 得到 ['10', '5'] (字符串列表)
# 2. map(int, ...) -> 将列表里的每个字符串都转换成 int
# 3. width, height = ... -> 将转换后的两个整数分别赋值

width, height = map(int, input_str.split())

area = width * height

print(f"The area is: {area}")
```

2.4.2 变量与赋值

在程序中，我们经常需要临时存储数据，以便后续使用。**变量 (Variable)** 就是用来存储数据的“容器”或“标签”。你可以把变量想象成一个带有名牌的储物盒，你可以把数据放进去，之后通过呼叫这个名牌来取用数据。

在 Python 中，创建一个变量并通过**赋值 (Assignment)** 操作将数据存入其中，语法非常简单，使用一个**等号 =** 即可。（注意：在编程语言中，相等一般用**双等号 ==** 表示）

```
# 创建一个名为 message 的变量，并把字符串 "Hello, Python!" 存进去

message = "Hello, Python!"

# 创建一个名为 student_count 的变量，并把整数 99 存进去

student_count = 99

# 创建一个名为 pi 的变量，并把浮点数 3.14 存进去

pi = 3.14
```

重要概念：赋值 vs. 相等

在编程中，单个**等号** `=` 表示**赋值**，意为“将右边的值，放入左边的变量中”。它是一个动作。

而我们在数学中常说的“**相等**”，在 Python 中用**双等号** `==` 表示，它用于比较左右两边的值是否相同，是一个判断，结果返回 `True` 或 `False`。我们将在后续章节中详细学习。

变量命名规则

给变量起一个好名字至关重要，它能让你的代码更易读。Python 的变量命名有几条规则：

- 只能包含字母、数字和下划线 `_`。
- 不能以数字开头。
- 不能使用 Python 的关键字（即有特殊含义的单词，如 `if`, `for`, `def` 等）。
- 变量名区分大小写 (`name` 和 `Name` 是两个不同的变量)。

推荐的命名风格 (Snake Case)

在 Python 中，社区推荐使用“蛇形命名法”，即所有字母小写，单词之间用下划线分隔。

```
# 好的命名
first_name = "Guido"
is_valid_user = True

# 不推荐的命名
FirstName = "Guido" # (这是驼峰命名法，在其他语言如 Java 中常用)
isValidUser = True
```

2.4.3 缩进

缩进 (Indentation) 是 Python 与其他编程语言（如 C, Java, JavaScript）最大的不同之处。

在其他语言中，通常使用大括号 `{}` 来表示代码块（比如 `if` 语句里面包含的代码）。但在 Python 中，我们使用“缩进”来表示代码的层级关系。

规则：同一层级的代码必须拥有相同长度的空白（空格）。

标准：虽然可以使用 Tab 键，但 Python 官方强烈建议使用 4 个空格作为一级缩进。（VS Code 等现代编辑器会自动帮你把 Tab 键转换成 4 个空格，你只需要按 Tab 即可）。

```
# 正确的缩进

if True:

    print("这句话在 if 内部") # 有缩进

    print("这句话也在 if 内部") # 缩进必须对齐

print("这句话在 if 外部")      # 没有缩进，顶格写
```

错误的缩进（会报错 IndentationError）

```
if True:

    print("第一行")

    print("第二行缩进不对齐，程序会崩溃！")
```

注意：初学者最容易遇到的错误就是 IndentationError，通常是因为多敲或少敲了空格。保持代码整洁对 Python 来说不是美德，而是强制要求。

2.4.4 注释

注释 (Comment) 是代码中不会被 Python 解释器执行的说明性文字。它们是写给自己或你的同事看的，用于解释某段代码的意图、逻辑或注意事项。编写清晰的注释是程序员的一个核心素养。

单行注释：以井号 # 开头，从 # 到行尾的所有内容都将被忽略。

```
# 这是一个单行注释，用来解释下面的代码

# 计算两个数的和

a = 5

b = 10

result = a + b # 也可以在代码行的末尾添加注释
```

多行注释：虽然 Python 没有专门的多行注释语法，但通常使用三个引号 '''...''' 或 """...""" 来实现多行文本块，常用于解释复杂的逻辑或作为文档说明。

```
def some_function():

    '''
```

这是一个多行注释块。

你可以在这里写下更详细的说明，

例如一个函数的用途、参数等。

```
'''
```

```
pass
```

2.5 Python 数据类型与结构

程序的核心是处理数据。数据可以是任何东西：一个用户的年龄、一篇新闻的标题、一张图片的所有像素值，或者是一个机器学习模型的配置参数。为了在 Python 中有效地表示和操作这些信息，我们需要了解它的**数据类型**和**数据结构**。

本节我们将学习 Python 中最基础、最常用的数据类型与结构。

2.5.1 基础类型

基础数据类型是构成所有复杂数据结构的最基本元素。

1. 数字 (Numbers)

整数 (Integer, int): 没有小数部分的数字，可以是正数、负数或零。

```
user_age = 25
```

```
year = -2023
```

```
count = 0
```

浮点数 (Float, float): 带有小数部分的数字。

```
pi_approx = 3.14159
```

```
model_accuracy = 0.985
```

```
temperature = -10.5
```

基本运算: Python 支持所有标准的数学运算。

```
a = 10
```

```
b = 3
```

```
print(a + b) # 加法 -> 13
```

```
print(a - b) # 减法 -> 7
```

```
print(a * b) # 乘法 -> 30
```

```
print(a / b) # 除法 -> 3.333...
```

```
print(a // b) # 整除 -> 3
```

```
print(a % b) # 取余 -> 1
```

```
print(a ** b) # 幂运算 -> 1000
```

2. 字符串 (String, str)

字符串是用于表示文本数据的一串字符序列，必须用单引号 `'...'`、双引号 `"..."` 或三引号 `'''...'''` 包围。

```
# 单引号和双引号没有区别
```

```
book_title = "Artificial Intelligence: A Modern Approach"
```

```
author = 'Stuart Russell'
```

```
# 三引号可以用来写多行字符串
```

```
long_text = '''
```

```
This is a multi-line string.
```

```
It is often used for documentation or long text blocks.
```

```
'''
```

```
# f-string 也可以用于高效地格式化字符串
```

```
name = "Alice"
```

```
age = 30
```

```
greeting = f"Hello, my name is {name} and I am {age} years old."
```

```
print(greeting) # -> Hello, my name is Alice and I am 30 years old.
```

字符串有很多内置的“方法”(method)来处理文本，我们将在后续章节深入探讨。

3. 布尔值 (Boolean, bool)

布尔值是逻辑世界的基础，它只有两个值：True (真) 和 False (假)。程序通过布尔值来进行判断和决策。

布尔值通常由**比较运算**和**逻辑运算**产生：

```
# 比较运算
```

```
print(5 > 3)          # -> True, 因为 5 大于 3
```

```
print(10 == 20)      # -> False (注意: == 是比较是否相等, = 是赋值)
```

```
print("python" != "java") # -> True, 因为两个字符串不相等
```

```
# 逻辑运算 (and, or, not)
```

```
print(True and False) # -> False (与运算: 两者都为真才为真)
```

```
print(True or False)  # -> True (或运算: 只要一个为真就为真)
```

```
print(not True)       # -> False (非运算: 取反)
```

布尔值的真实值

在 Python 内部, 布尔值实际上是整数 (int) 的一个“子类型”。True 在计算时等同于整数 1, 而 False 等同于 0。你可以通过简单的运算来验证这一点:

```
print(True == 1)      # -> True
```

```
print(False == 0)     # -> True
```

```
print(True + True)    # -> 2 (相当于 1 + 1)
```

```
print(5 * False)      # -> 0 (相当于 5 * 0)
```

理解这一点有助于解释为什么在某些情况下, 布尔值可以和数字混合运算。但在编写逻辑判断时, 我们应该始终明确地使用 True 和 False, 以保证代码的清晰可读。

布尔值是控制程序流程 (例如 if 语句) 的关键。

2.5.2 内置容器 (列表、字典、元组、集合)

容器用于存放多个数据项。Python 提供了四种非常强大的内置容器。

1. 列表 (List)

列表是 Python 中最常用的容器, 它是一个有序的、可变的元素集合, 你可以按顺序存取元素, 随时添加、删除或修改任何一项。

特点: 有序、可变、允许元素重复。

语法: 使用方括号 [] 定义, 元素间用逗号 , 分隔。

```
# 一个包含不同类型元素的列表
```

```
features = ["age", 25, 0.95, True]
```

```
# 访问元素 (索引从 0 开始)
```

```
print(features[0]) # -> "age"
```

```
print(features[-1]) # -> True (负数索引表示从末尾开始)
```

```
# 修改元素
```

```
features[1] = 26
```

```
print(features)      # -> ['age', 26, 0.95, True]
```

```
# 添加元素
```

```
features.append("new_feature")
```

```
print(features)      # -> ['age', 26, 0.95, True, 'new_feature']
```

```
# 删除元素
```

```
features.pop() # 删除最后一个元素
```

```
print(features)      # -> ['age', 26, 0.95, True]
```

```
# 获取列表长度
```

```
print(len(features)) # -> 4
```

2. 元组 (Tuple)

元组可以看作是一个不可变的列表。其中的元素一旦确定，就不可再改变它。

特点：有序、不可变、允许元素重复。

语法：使用圆括号 () 定义。

```
# 定义一个元组
```

```
point = (120.15, 30.28)
```

```
# 访问元素
```

```
print(point[0]) # -> 120.15
```

```
# 尝试修改元组会报错！
```

```
# point[0] = 121.0 # 这会引发 TypeError
```

元组的主要用途是保证数据的完整性，不被意外修改

3. 字典 (Dictionary, dict)

字典是一个无序的 (在 Python 3.7+ 中为插入有序)、可变的键值对 (key: value) 集合。字典就像一本真正的英文字典，你可以通过“单词” (key) 快速查到它的“释义”(value)，而不用从第一页翻到最后一页。

特点: 键值对存储、键必须唯一且不可变、可变。

语法: 使用花括号 {} 定义。

描述一个 AI 模型的配置文件

```
model_config = {  
    "model_name": "ResNet50",  
    "learning_rate": 0.001,  
    "layers": [50, 100, 50],  
    "is_training": True  
}
```

通过 key 访问 value

```
print(model_config["model_name"]) # -> "ResNet50"
```

修改/添加键值对

```
model_config["learning_rate"] = 0.002 # 修改
```

```
model_config["optimizer"] = "Adam" # 新增
```

```
print(model_config)
```

获取所有键

```
print(model_config.keys()) # -> dict_keys(['model_name', ...])
```

获取所有值

```
print(model_config.values()) # -> dict_values(['ResNet50', ...])
```

4. 集合 (Set)

集合是一个无序的、不含重复元素的容器。类比数学中集合的概念。

特点: 元素唯一、无序、可变。

语法: 使用花括号 {} 或 set() 函数定义。

```
# 定义一个有重复元素的列表
tags_list = ["python", "ai", "data", "python", "ml"]

# 转换成集合以去重
unique_tags = set(tags_list)
print(unique_tags) # -> {'data', 'ml', 'python', 'ai'} (顺序可能不同)

# 集合的主要用途是去重和进行数学运算
set_a = {1, 2, 3, 4}
set_b = {3, 4, 5, 6}

# 交集 (共同拥有的元素)
print(set_a & set_b) # -> {3, 4}

# 并集 (所有不重复的元素)
print(set_a | set_b) # -> {1, 2, 3, 4, 5, 6}
```

2.5.3 切片与推导式

1. 切片 (Slicing)

切片是一种快速提取序列（如列表、元组、字符串）一部分的语法。

语法: sequence[start:stop:step] (start 包含, stop 不包含)

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

print(numbers[2:5]) # -> [2, 3, 4] (从索引 2 到 5, 不含 5)
print(numbers[:4]) # -> [0, 1, 2, 3] (从开头到 4)
print(numbers[5:]) # -> [5, 6, 7, 8, 9] (从 5 到结尾)
print(numbers[::2]) # -> [0, 2, 4, 6, 8] (从头到尾, 步长为 2)
print(numbers[::-1]) # -> [9, 8, 7, 6, 5, 4, 3, 2, 1, 0] (巧妙地翻转序列)
```

2. 推导式 (Comprehensions)

推导式是一种简洁、高效地从一个现有容器创建新容器的“Pythonic”方式。

a. 列表推导式 (List Comprehension)

假设我们想创建一个包含 0-9 所有数字平方的列表。

传统方法：

```
squares = []  
  
for i in range(10):  
    squares.append(i ** 2)  
  
# squares -> [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

推导式方法：

```
squares = [i ** 2 for i in range(10)]  
  
# squares -> [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

推导式将“做什么”(i ** 2)和“对谁做”(for i in range(10))写在了一行，代码更紧凑，可读性更强，执行效率也更高。

b. 字典和集合推导式

同样的语法也可以用于创建字典和集合。

```
# 字典推导式  
  
square_dict = {x: x**2 for x in range(5)}  
  
# -> {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}  
  
  
# 集合推导式  
  
unique_squares = {x**2 for x in [1, -1, 2, -2]}  
  
# -> {1, 4}
```

2.5.4 可变 vs 不可变、浅拷贝 vs 深拷贝

这是一个 Python 中非常重要但初学者容易混淆的概念。

1. 可变 (Mutable) vs. 不可变 (Immutable)

a. **不可变类型**：一旦被创建，其值就不能被改变。如果你尝试“修改”，实际上是创建了一个新的对象。

包括：数字 (int, float)，字符串 (str)，布尔 (bool)，元组 (tuple)

```
x = 10
```


id() 函数可以获取对象在内存中的唯一地址

```
print(id(x)) # -> 140702...
```

`x = x + 1` # 这里不是修改了 10，而是创建了一个新的对象 11，并让 x 指向它

```
print(id(x)) # -> 140702... (地址变了)
```

b. 可变类型: 创建后，其值可以被改变，而不需要创建新对象。

包括：列表 (list), 字典 (dict), 集合 (set)

```
my_list = [1, 2, 3]
```

```
print(id(my_list)) # -> 22895...
```

```
my_list.append(4) # 这里是原地修改了列表对象
```

```
print(my_list) # -> [1, 2, 3, 4]
```

```
print(id(my_list)) # -> 22895... (地址没变)
```

理解这一点对于避免函数中意外的“副作用”至关重要。

2. 浅拷贝 (Shallow Copy) vs. 深拷贝 (Deep Copy)

这个问题源于当可变对象中嵌套了其他可变对象时。

a. 赋值 =: 只是创建了一个新的“标签”（变量名），指向同一个内存地址。

```
a = [1, 2, [10, 20]]
```

```
b = a # b 只是 a 的一个别名
```

```
b[0] = 99
```

```
print(a) # -> [99, 2, [10, 20]] (修改 b 会影响 a)
```

b. 浅拷贝 (.copy() 或 [:]): 创建了一个新的顶层容器，但其内部的元素如果是可变对象，则只拷贝其引用（地址）。

类比：复印了一份文件清单，清单本身是新的，但清单上列出的文件（如果是文件夹）还是指向原来的那个文件夹。

```
import copy
```

```
a = [1, 2, [10, 20]]
```

```
c = a.copy() # 或 c = a[:]
```

```
c[0] = 88 # 修改顶层元素，不影响 a
```

```
print(a) # -> [1, 2, [10, 20]]
```

```
c[2][0] = 999 # 修改嵌套列表的元素，会影响 a!
```

```
print(a) # -> [1, 2, [999, 20]]
```

c. 深拷贝 (copy.deepcopy()): 完全创建一个新的、独立的副本，包括所有嵌套的对象。

类比：复印了文件清单，并且把清单上列出的所有文件夹及其内容也全部复印了一份。

```
import copy
```

```
a = [1, 2, [10, 20]]
```

```
d = copy.deepcopy(a)
```

```
d[2][0] = 777 # 修改嵌套列表的元素，完全不影响 a
```

```
print(a) # -> [1, 2, [10, 20]]
```

```
print(d) # -> [1, 2, [777, 20]]
```

在处理复杂的数据结构（如多层嵌套的字典或列表）时，必须清楚你需要的拷贝类型，以避免数据污染。

2.6 程序控制流程。

如果把编写程序比作铺设一条铁路，那么此前我们要么是在造铁轨（数据结构），要么是在造火车（变量）。而本节要讲的，是道岔和信号灯。

默认情况下，Python 代码是从上到下、一行接一行顺序执行的。但在实际应用中，我们需要程序能够根据不同的情况走不同的路径（条件判断），或者重复执行某项任务（循环）。这就是程序控制流程。

2.6.1 条件判断语句 (if / match)

1. if 语句：让程序学会二选一

这是编程中最基础的逻辑。我们要告诉计算机：“如果满足这个条件，就做这件事；否则，做那件事。”

Python 使用 if、elif (else if 的缩写) 和 else 关键字。

```

score = 85

if score >= 90:
    print("优秀! (Grade A)")
elif score >= 75:
    print("良好。 (Grade B)")
elif score >= 60:
    print("及格。 (Grade C)")
else:
    print("不及格，请继续努力。")

```

核心概念：真值测试 (Truth Value Testing)

还记得我们在 3.5.1 节讨论布尔值时留下的伏笔吗？在 `if` 后面的条件判断中，Python 并不强制要求你必须提供一个 `True` 或 `False`。

Python 会对 `if` 后面的任何对象进行“真值测试”。规则非常简单且符合直觉：

被视为“假” (`False`) 的值：0, 0.0, `None`, `False`, 空的容器（空字符串 `""`，空列表 `[]`，空字典 `{}` 等）。

被视为“真” (`True`) 的值：除上述以外的所有值。

这让我们的代码可以写得非常简洁：

```

# 场景：判断用户是否输入了名字

user_name = input("请输入你的名字: ")

# 不推荐写法： if len(user_name) > 0:
# 推荐写法： 直接判断字符串对象

if user_name:
    print(f"你好, {user_name}!")
else:
    print("你没有输入名字。")

# 场景：处理待办事项列表

```

```

todo_list = ["Buy milk", "Walk the dog"]

if todo_list:

    print(f'你还有 {len(todo_list)} 件事要做。')

```

2. match 语句：结构化模式匹配 (Python 3.10+)

在 Python 3.10 之前，如果我们有多个固定的值需要判断，只能写一连串的 if-elif-elif。现在，我们有了更优雅的 match 语句（类似于 C/Java 中的 switch，但更强大）。

这在编写系统指令解析或状态机时非常有用。

```

status = 404

match status:

    case 200:

        print("请求成功")

    case 400:

        print("错误请求")

    case 404:

        print("页面未找到")

    case 500 | 502: # 可以用 | 匹配多个值

        print("服务器错误")

    case _: # 下划线 _ 表示“匹配所有其他情况”，相当于 else

        print("未知状态码")

```

2.6.2 循环结构 (for / while / else / break / continue)

循环让程序能够处理大量重复的任务，比如“处理这 10000 张图片”或“训练模型直到收敛”。

1. for 循环：遍历一切

Python 的 for 循环与 C 或 Java 的 for(i=0; i<n; i++) 不同，它更像是一个“迭代器”。它可以直接遍历任何序列（列表、字符串、字典等）。

```

# 遍历列表

models = ["GPT-4", "BERT", "ResNet"]

for model in models:

```

```
print(f'正在加载模型: {model}')
```

```
# 使用 range() 生成数字序列
```

```
# range(start, stop, step)
```

```
for i in range(1, 10, 2):
```

```
    print(i) # 输出 1, 3, 5, 7, 9
```

2. while 循环：只要条件满足就一直跑

当我们要重复执行任务，但不知道具体要重复多少次，只知道停止条件时，用 while。

```
battery = 10
```

```
while battery > 0:
```

```
    print(f'当前电量 {battery}%, 正在工作...')
```

```
    battery -= 5 # 每次循环电量减 5
```

```
print("电量耗尽，系统关机。")
```

3. 控制循环的指挥棒：break 和 continue

break: 立即终止整个循环，跳出圈外。

continue: 立即跳过本次循环剩余的代码，直接开始下一次循环。

```
# 任务：在数据中寻找目标，忽略错误数据
```

```
data_stream = [10, 20, -1, 30, 999, 40] # -1 表示坏数据，999 表示结束信号
```

```
for data in data_stream:
```

```
    if data == 999:
```

```
        print("收到结束信号，停止处理。")
```

```
        break # 彻底结束循环
```

```
    if data < 0:
```

```
        print("发现坏数据，跳过。")
```

```
        continue # 跳过 print(data)，直接处理下一个
```

```
print(f'正在处理数据: {data}')
```

4. 循环中的 else 子句: Python 的特色

Python 的循环竟然可以带一个 **else**! 这常常让初学者困惑。

这里的 **else** 的含义是: “如果没有被 **break** 打断, 正常执行完所有循环后, 执行这里。”

它常用于“搜索”场景:

```
# 任务: 判断一个数是否为素数
```

```
num = 13
```

```
for i in range(2, num):
```

```
    if num % i == 0:
```

```
        print(f'{num} 不是素数, 它能被 {i} 整除。")
```

```
        break # 找到了因子, 任务结束, 强行打断循环
```

```
    else:
```

```
        # 只有当循环正常跑完 (即 2 到 num-1 都试过了, 没触发 break), 才执行这里
```

```
        print(f'{num} 是一个素数。")
```

2.6.3 迭代协议简析 (Iteration Protocol)

注意: 本小节涉及 Python 运行机制的底层原理, 初学者只需理解概念即可。

你是否好奇过, 为什么 **for** 循环可以遍历列表、字符串, 甚至文件? 为什么它们统称为“可迭代对象”(Iterable)?

这背后是一套被称为迭代协议的规则。

(1) 可迭代对象 (Iterable):

这就好比一个“**播放列表**”。它里面包含数据, 但它自己不知道当前播放到哪一首了。

常见的 `list`, `tuple`, `str` 都是 Iterable。

(2) 迭代器 (Iterator):

这就好比播放器的**“下一首”按钮**。它记录了当前的播放进度。每次你按一下 (请求数据), 它就给你吐出下一个元素。

1. for 循环的幕后真相

当你写 `for x in my_list:` 时，Python 悄悄做了两件事：

获取迭代器：它先找 `my_list` 要了一个迭代器（相当于拿到了播放控制器）。

不断索取：它不断地按“下一首”按钮，把拿到的数据赋值给 `x`，直到没有更多数据为止。

我们可以用代码手动模拟这个过程：

```
# 一个普通的列表（可迭代对象）
```

```
fruits = ["apple", "banana"]
```

```
# 1. 手动获取它的迭代器
```

```
iterator = iter(fruits)
```

```
# 2. 手动获取数据
```

```
print(next(iterator)) # -> apple
```

```
print(next(iterator)) # -> banana
```

```
# 3. 再取一次？
```

```
# print(next(iterator)) # -> 报错！StopIteration
```

为什么要了解这个？

在 AI 训练中，数据集往往非常大（比如几 TB 的图片），我们不可能一次性把它们全部读入内存（塞进一个列表）。

利用迭代协议，我们可以创建一个“**懒惰**”的迭代器：每次训练需要数据时，它才去硬盘读取一张图片。这样，无论数据多大，内存占用都非常小。这将在第 4 章的“生成器”部分详细讲解。

2.7 函数基础

如果说变量是数据的容器，控制流是程序的骨架，那么函数 (Function) 就是程序的肌肉。

函数将一段具有特定功能的代码“打包”在一起，起个名字，方便在任何地方重复调用。这体现了编程中最重要的原则之一：DRY (Don't Repeat Yourself, 不要重复自己)。

在 AI 开发中，你可能会定义一个函数来读取图片，定义另一个函数来清洗数据，再定义一个函数来计算准确率。将复杂的问题拆解为一个个小的函数，是构建大型智能系统的基础。

2.7.1 定义与参数 (位置、关键字、变参)

1. 函数的定义

Python 使用 `def` 关键字来定义函数。

```
# 定义一个计算 ReLU 激活函数的简单函数

# ReLU (Rectified Linear Unit) 是深度学习中最常用的激活函数

# 规则：如果  $x > 0$ ，返回  $x$ ；否则返回 0
```

```
def relu(x):
    """
    计算 ReLU 激活值

    :param x: 输入数值
    :return: 激活后的数值
    """
    if x > 0:
        return x
    else:
        return 0
```

```
# 调用函数

result = relu(-5)

print(result)  # -> 0
```

2. 参数传递的三种姿势

Python 的参数传递非常灵活，主要有以下几种方式：

(1) **位置参数 (Positional Arguments)**: 最常见的方式，按照定义的顺序依次传递。

```
def describe_model(name, layers):
```



```
print(f'模型 {name} 拥有 {layers} 层神经网络。")
```

```
describe_model("ResNet", 50)
```

-> 模型 ResNet 拥有 50 层神经网络。

(2) 关键字参数 (Keyword Arguments): 通过“键=值”的形式传递，不需要死记参数顺序，代码可读性极高。

顺序乱了也没关系

```
describe_model(layers=101, name="DenseNet")
```

-> 模型 DenseNet 拥有 101 层神经网络。

(3) 默认参数 (Default Arguments): 为参数提供一个默认值。如果不传该参数，就使用默认值。这在配置机器学习模型的超参数时非常有用。

```
def train_model(epochs=10, learning_rate=0.01):
```

```
    print(f'训练轮数: {epochs}, 学习率: {learning_rate}')
```

```
train_model()          # 使用全部默认值
```

```
train_model(epochs=50) # 只修改轮数
```

```
train_model(learning_rate=0.001) # 只修改学习率
```

(4) 警惕：可变默认参数的陷阱

这是一个 90% 的 Python 新手（甚至老手）都会踩的坑！永远不要使用可变对象（如列表、字典）作为参数的默认值。

错误示范

```
def append_data(new_item, data_list=[]):
```

```
    data_list.append(new_item)
```

```
    return data_list
```

```
print(append_data(1)) # -> [1] (看起来正常)
```

```
print(append_data(2)) # -> [1, 2] (?? 为什么 1 还在?)
```

原因：函数的默认值在函数定义时就被创建了，并且只创建一次。后续所有调用都共享同一个列表对象。

正确做法：使用 None 作为默认值。

```
def append_data(new_item, data_list=None):  
    if data_list is None:  
        data_list = []  
    data_list.append(new_item)  
    return data_list
```

3. 可变参数 (*args 与 **kwargs)

当你不确定调用者会传入多少个参数时，这两个神器就派上用场了。在阅读开源 AI 框架源码时，你会经常看到它们。

(1) ***args (Arguments)**: 将多余的位置参数打包成一个元组 (Tuple)。

```
# 计算任意多个数字的平均值  
def calculate_average(*numbers):  
    # numbers 是一个元组，例如 (10, 20, 30)  
    if not numbers:  
        return 0  
    return sum(numbers) / len(numbers)
```

```
print(calculate_average(10, 20, 30)) # -> 20.0
```

(2) ****kwargs (Keyword Arguments)**: 将多余的关键字参数打包成一个字典 (Dict)。

这常用于传递模型的配置信息。

```
def create_layer(layer_type, **config):  
    print(f"正在创建 {layer_type} 层...")  
    # config 是一个字典，例如 {'units': 64, 'activation': 'relu'}  
    for key, value in config.items():  
        print(f"配置 {key}: {value}")  
  
create_layer("Dense", units=64, activation="relu", use_bias=True)
```

2.7.2 返回值、作用域、闭包与 lambda

1. 返回值 (Return Values)

Python 函数可以返回一个值，也可以通过元组同时返回多个值。如果省略 `return`，函数默认返回 `None`。

```
def evaluate_model(y_true, y_pred):  
    # 假设经过复杂的计算...  
  
    loss = 0.05  
  
    accuracy = 0.98  
  
    # 返回多个值，实际上是返回了一个元组 (0.05, 0.98)  
  
    return loss, accuracy
```

```
# 自动解包 (Unpacking)  
  
l, acc = evaluate_model(target, prediction)  
  
print(f'损失: {l}, 准确率: {acc}')
```

2. 作用域 (Scope) 与 LEGB 规则

变量并不是哪里都能访问的。变量起作用的范围称为作用域。

局部变量 (Local): 在函数内部定义的变量。出了函数就失效了，“在拉斯维加斯发生的事，留在拉斯维加斯”。

全局变量 (Global): 在函数外部定义的变量。

```
total_epochs = 100 # 全局变量
```

```
def train():  
    current_epoch = 5 # 局部变量  
  
    print(total_epochs) # 可以读取全局变量  
  
    # total_epochs = 200 # 注意：这会创建一个新的局部变量，而不是修改全局  
    变量！
```

```
# 如果非要修改全局变量（通常不推荐），需要用 global 关键字  
  
global total_epochs  
  
total_epochs = 200
```

```
train()
```

```
# print(current_epoch) # 报错！ 外部无法访问函数内部变量
```

3. 闭包 (Closure)

闭包是 Python 中较高级但也非常强大的概念。简单来说，闭包就是一个能记住其定义时环境（外部变量）的函数。

它常用于“工厂模式”，即编写一个“生产函数的函数”。

场景：我们要生成不同的线性函数 $y=kx+b$ 。

```
def make_linear_function(k, b):
```

```
    # 这是一个内部函数，它用到了外部的 k 和 b
```

```
    def linear(x):
```

```
        return k * x + b
```

```
    # 返回这个内部函数
```

```
    return linear
```

```
# 创建一个  $y = 2x + 1$  的函数
```

```
f1 = make_linear_function(2, 1)
```

```
# 创建一个  $y = 5x + 10$  的函数
```

```
f2 = make_linear_function(5, 10)
```

```
print(f1(5)) # -> 11 (2*5 + 1)
```

```
print(f2(5)) # -> 35 (5*5 + 10)
```

在这里，f1 就是一个闭包，它虽然已经脱离了 make_linear_function，但它依然“记得”
k=2 和 b=1。

4. 匿名函数 (Lambda Expressions)

有些时候，我们需要一个非常简单的函数（比如只做一次加法），专门为此写个 def 有点小题大做。这时可以使用 lambda 关键字来创建匿名函数。

语法: lambda 参数: 表达式

应用: 常配合 sort (排序)、map (映射) 等高阶函数使用。

```
# 普通函数
```

```
def square(x):
    return x ** 2

# 等价的 Lambda 函数
square_lambda = lambda x: x ** 2

print(square_lambda(5)) # -> 25

# --- 实战场景：按字典中的某个值排序 ---
students = [
    {"name": "Alice", "score": 88},
    {"name": "Bob", "score": 95},
    {"name": "Charlie", "score": 70}
]

# 按分数 (score) 从高到低排序
# key 参数需要一个函数，这里直接原地写一个 lambda
students.sort(key=lambda x: x["score"], reverse=True)

print(students)

# -> Bob 排第一，Alice 排第二...
```

理解了函数，你就掌握了封装逻辑的能力。

2.8 模块与包 (基础)

当你的代码从几十行变成几百行、几千行时，你不可能把所有逻辑都写在一个文件里。那就像把所有的书都堆在地上，而不是放在书架上一样混乱。

Python 提供了模块 (Module) 和 包 (Package) 的机制，帮助我们组织代码。这也是为什么你可以通过简单的 `import torch` 就使用 Facebook 工程师写好的几百万行深度学习代码的原因。

2.8.1 模块概念与导入机制

1. 什么是模块？

很简单，每一个以 `.py` 结尾的 Python 文件就是一个模块。文件名就是模块名。

假设我们写了一个名为 `metrics.py` 的文件，里面定义了一些计算误差的函数：

```
# 文件名: metrics.py

def calculate_accuracy(y_true, y_pred):

    """计算准确率"""

    return sum(t == p for t, p in zip(y_true, y_pred)) / len(y_true)


def calculate_mse(y_true, y_pred):

    """计算均方误差"""

    error = sum((t - p) ** 2 for t, p in zip(y_true, y_pred))

    return error / len(y_true)
```

2. 如何导入模块？

在另一个文件（比如 `main.py`）中，我们可以通过 `import` 关键字来使用它。

方式一：直接导入 (`import`)

```
import metrics

acc = metrics.calculate_accuracy([1, 0], [1, 1])
```

方式二：导入并起别名 (`import ... as ...`)

这是 AI 领域最常用 的方式。比如我们几乎总是把 NumPy 简写为 `np`，把 Pandas 简写为 `pd`。

```
import metrics as mt # 给模块起个短名字

acc = mt.calculate_accuracy([1, 0], [1, 1])

# 常见的 AI 惯例：

# import numpy as np
```

```
# import pandas as pd

# import matplotlib.pyplot as plt

# import torch.nn as nn
```

方式三：导入特定函数 (from ... import ...)

如果你只需要模块中的某个函数，不想导入整个模块。

```
from metrics import calculate_accuracy

# 可以直接使用函数名，不需要加模块前缀

acc = calculate_accuracy([1, 0], [1, 1])
```

警告：避免使用 `from module import *`

这种写法会将模块里所有的东西都倒进你的命名空间，很容易导致变量名冲突（比如模块里有个变量叫 `loss`，你自己的代码里也有个 `loss`，会被覆盖）。显式优于隐式，请尽量明确你导入了什么。

3. if __name__ == '__main__':

在阅读开源代码时，你经常会在文件末尾看到这句话。它的作用是：区分“作为脚本运行”和“作为模块导入”。

当你在终端直接运行 `python metrics.py` 时，Python 会把内部变量 `__name__` 赋值为 `'__main__'`。

当你 `import metrics` 时，`__name__` 的值就是 `'metrics'`。

metrics.py 的末尾

```
if __name__ == '__main__':
    # 只有直接运行此文件时，才会执行这里的测试代码
    # 如果被别的文件 import，这里不会执行
    print("正在测试 metrics 模块...")
    test_acc = calculate_accuracy([1, 1], [1, 0])
    print(f'测试结果: {test_acc}')
```

2.8.2 包与路径管理

1. 什么是包？

如果模块是“文件”，那么包就是“文件夹”。

包 (Package) 是一个包含多个模块的目录。为了让 Python 知道这个目录是一个包，通常（但不强制要求，视 Python 版本而定，但建议保留）需要在目录下放一个名为 `__init__.py` 的文件。

2. 项目结构示例

一个典型的 AI 项目结构如下：

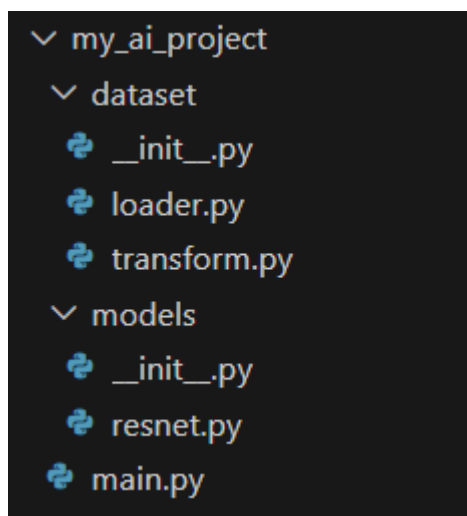


图 3.20 AI 项目结构实例

3. 如何跨文件夹导入？

在 `main.py` 中，我们可以使用点号 `.` 来表示层级关系。

```
# main.py

# 从 dataset 包下的 loader 模块导入 load_image 函数
from dataset.loader import load_image

# 导入整个 models 包下的 resnet 模块
import models.resnet as net

img = load_image("cat.jpg")

model = net.ResNet50()
```

4. `__init__.py` 的作用

当一个包被导入时，`__init__.py` 中的代码会自动执行。

你可以利用它来简化导入路径。例如，如果在 `dataset/__init__.py` 中写了：

```
# dataset/__init__.py

from .loader import load_image

from .transform import resize_image
```

那么在外面就可以直接从包导入，而不需要指定模块文件：

```
# 更加简洁

from dataset import load_image, resize_image
```

2.8.3 标准库速览 (Batteries Included)

Python 被称为“自带电池”的语言，因为它内置了极其丰富的标准库。在安装第三方库（如 PyTorch）之前，你应该先看看标准库里有没有现成的工具。

以下是 AI 开发中最常用的几个标准库：

1. os 和 sys：操作系统与系统交互

虽然我们后面会讲更现代的 `pathlib`，但 `os` 依然是基础。

```
import os

import sys

# 获取当前工作目录

print(os.getcwd())

# 检查文件是否存在 (非常常用，比如检查模型权重文件是否下载)

if os.path.exists("weights.pt"):

    print("加载模型...")

else:

    print("错误：权重文件不存在！")

# sys.path: Python 搜索模块的路径列表

# 如果你 import 报错找不到模块，打印它看看你的路径对不对

print(sys.path)
```

2. time 和 datetime: 时间与日期

用于计算模型训练耗时，或者给日志文件打上时间戳。

```
import time

from datetime import datetime

# 记录开始时间

start_time = time.time()


# 模拟训练过程

time.sleep(2) # 暂停 2 秒


# 计算耗时

duration = time.time() - start_time

print(f'训练耗时: {duration:.2f} 秒')


# 获取当前格式化时间

now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

print(f'日志时间: {now}')
```

3. random: 随机数生成

虽然 AI 训练通常用 NumPy 的随机数，但 Python 自带的 random 依然常用于简单的打乱数据。

```
import random


data = ["img1.jpg", "img2.jpg", "img3.jpg"]


# 随机打乱列表 (原地修改)

random.shuffle(data)

print(data)
```

```
# 随机采样一个  
  
chosen = random.choice(data)
```

4. json: 处理 JSON 数据

在 AI 任务中，配置文件和标注文件（如 COCO 数据集）通常是 JSON 格式。这部分我们将在下一节的文件 I/O 中详细实战。

2.9 文件与 I/O

在人工智能领域，程序永远不会孤立运行。你需要读取大量的数据集（图片、文本、表格），训练模型，然后将训练日志和模型参数保存到硬盘上。文件 I/O (输入/输出) 操作，是数据流动的管道。

Python 提供了极其简洁且强大的文件处理接口。本节我们将学习如何安全、高效地读写文件，以及如何像处理对象一样优雅地处理文件路径。

2.9.1 open() 与 with 上下文管理

1. 文件的打开与关闭

要操作一个文件，首先需要“打开”它。Python 内置的 `open()` 函数用于创建文件句柄。

最原始（但不推荐）的写法是这样的：

```
f = open("log.txt", "w") # 'w' 表示写入模式 (Write)  
  
f.write("Training started...")  
  
# 如果在这里程序发生了错误（抛出异常），下一行的 close() 将永远不会执行！  
# 这会导致文件资源泄露，甚至数据丢失。  
  
f.close()
```

2. 推荐写法：with 语句

为了保证文件一定会被正确关闭（即使中间发生了错误），Python 引入了 上下文管理器 (Context Manager)，也就是 `with` 关键字。

```
# 'w': 写入模式 (会覆盖原有内容)  
  
# 'a': 追加模式 (在末尾添加内容)  
  
# 'r': 读取模式 (默认)
```

```
# 进入 with 代码块时，文件自动打开

with open("log.txt", "w", encoding="utf-8") as f:

    f.write("Epoch 1: Loss 0.5\n")

    f.write("Epoch 2: Loss 0.3\n")

# 离开 with 代码块（或发生异常）时，f.close() 会被自动调用

# 此时文件已安全关闭
```

2.9.2 文本 vs. 二进制文件读取

计算机里的文件主要分为两类：给人看的（文本）和给机器看的（二进制）。

文本模式 (t): 默认模式。Python 会自动将文件内容解码为字符串 (str)。适用于 .txt, .csv, .py, .json 等。

二进制模式 (b): 读取原始字节 (bytes)。适用于图片 (.jpg), 音频 (.wav), 视频, 以及模型权重文件 (.pkl, .pt)。

1. 读取文本文件

```
with open("log.txt", "r", encoding="utf-8") as f:

    content = f.read() # 一次性读取所有内容

    print(content)
```

2. 读取二进制文件 (例如一张图片)

注意模式是 'rb' (Read Binary)

```
with open("cat.jpg", "rb") as f:

    data = f.read(10) # 只读取前 10 个字节

    print(data)

# 输出示例: b'\xff\xd8\xff\xe0\x00\x10JFIF' (这是 JPEG 文件的文件头)
```

2.9.3 编码与错误处理 (UTF-8)

在处理包含中文、日文或特殊符号的文本数据时，编码 (Encoding) 是一个绕不开的坑。

痛点：Windows 中文版默认使用 GBK 编码，而 Linux/macOS 和现代互联网标准默认使用 UTF-8。如果你在 Windows 上保存了一个文件，拿到 Linux 服务器上训练，不指定编码就会出现乱码（Mojibake）。

黄金法则：永远显式指定 `encoding='utf-8'`。

正确做法：跨平台无忧

```
with open("data_zh.txt", "w", encoding="utf-8") as f:
```

```
    f.write("人工智能")
```

错误做法：依赖系统默认编码

在 Windows 上可能是 GBK，在 Linux 上是 UTF-8，导致互不兼容

```
with open("data_zh.txt", "w") as f:
```

```
    pass
```

2.9.4 CSV / JSON 基础处理

在 AI 项目中，我们很少直接读写 .txt，更多的是处理结构化数据。

1. JSON (JavaScript Object Notation)

JSON 是最常用的配置和轻量级数据交换格式。它和 Python 的字典/列表结构几乎一一对应。

场景：保存模型超参数 (config.json)，保存 COCO 数据集标注。

```
import json
```

定义训练配置 (Python 字典)

```
hyperparameters = {  
    "learning_rate": 0.001,  
    "batch_size": 32,  
    "optimizer": "Adam",  
    "layers": [128, 64, 10]  
}
```

1. 将字典保存为 JSON 文件 (Dump)

```
with open("config.json", "w", encoding="utf-8") as f:
    # indent=4 让文件格式化缩进，便于人类阅读
    json.dump(hyperparameters, f, indent=4)
```

2. 从 JSON 文件读取配置 (Load)

```
with open("config.json", "r", encoding="utf-8") as f:
    loaded_config = json.load(f)
```

```
print(loaded_config["batch_size"]) # -> 32
```

2. CSV (Comma-Separated Values)

CSV 是最通用的表格数据格式。虽然在数据科学中我们通常使用 **Pandas** 库来处理 CSV（将在后续进阶课程介绍），但了解 Python 标准库的处理方式依然很有必要。

场景：读取房价数据集，保存训练过程中的 loss 记录。

```
import csv

# 写入 CSV
with open("train_log.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["epoch", "loss", "accuracy"]) # 写表头
    writer.writerow([1, 0.52, 0.88])
    writer.writerow([2, 0.41, 0.92])

# 读取 CSV
with open("train_log.csv", "r", encoding="utf-8") as f:
    reader = csv.reader(f)
    for row in reader:
        print(row) # -> ['epoch', 'loss', 'accuracy'] ...
```

2.9.5 使用 **pathlib** 操作路径与遍历

在过去，Python 程序员使用 `os.path.join` 来凭借文件路径，这非常繁琐且容易出错（Windows 用 `\`，Linux 用 `/`）。

从 Python 3.4 开始，`pathlib` 模块提供了一种面向对象的、现代化的路径处理方式。这是 AI 工程中的最佳实践。

1. 路径的拼接与创建

```
from pathlib import Path

# 定义基础路径（假设是当前目录下的 datasets 文件夹）
base_dir = Path("./datasets")

# 使用 / 运算符拼接路径！就像在写 URL 一样自然
# 自动适配 Windows/Linux 分隔符
images_dir = base_dir / "cats_vs_dogs" / "train"

# 检查路径是否存在
if not images_dir.exists():
    # 创建文件夹（parents=True 表示自动创建父级目录，exist_ok=True 表示已
    # 存在不报错）
    images_dir.mkdir(parents=True, exist_ok=True)
    print(f'创建目录: {images_dir}')
```

2. 遍历文件（AI 数据集读取核心）

假设我们需要找到文件夹下所有的 `.jpg` 图片来制作数据集。

```
# glob: 通配符搜索
# rglob: 递归搜索 (Recursive glob)，会在所有子文件夹中查找

print(f'正在搜索 {images_dir} 下的图片...')

# 找到所有的 .jpg 文件
image_files = list(images_dir.glob("*.jpg"))
```

```
# 或者递归查找所有子目录下的 .jpg
all_images = list(base_dir.rglob("*.jpg"))

for img_path in image_files[:5]: # 只打印前 5 个
    # .name 获取文件名, .stem 获取文件名(不含后缀), .suffix 获取后缀
    print(f'发现图片: {img_path.name}')
    # 还可以直接转为字符串传给 open() 或 cv2.imread()
    # str(img_path)
```

2.10 异常、调试与单元测试

编程有句名言：“代码写完的那一刻，Bug 就已经存在了。”

无论你的水平有多高，错误（Bugs）都是不可避免的。可能是除以了零，可能是文件找不到，也可能是网络超时。本节我们将学习如何优雅地处理这些意外（异常处理），如何像侦探一样找到错误的根源（调试），以及如何为代码穿上一层防弹衣（单元测试）。

2.10.1 异常处理结构 (try / except 等)

1. 什么是异常 (Exception)?

当 Python 代码运行过程中遇到错误时，它会停止执行并抛出一个“异常”。如果不处理，程序就会直接崩溃（Crash）。

常见的异常包括：

- **ZeroDivisionError**: 除以零。
- **FileNotFoundError**: 试图打开不存在的文件。
- **IndexError**: 访问列表越界。
- **KeyError**: 字典里没有这个键。
- **TypeError**: 类型不匹配（比如把字符串和数字相加）。

2. 捕获异常：try ... except

我们可以使用 try 语句块来“包裹”可能出错的代码。

AI 场景：在读取海量图片进行训练时，如果某张图片文件损坏了，我们希望跳过它继续训练，而不是让整个程序崩溃。


```

image_paths = ["img1.jpg", "corrupt.jpg", "img3.jpg"]

for img_path in image_paths:
    try:
        # 假设这是一个读取图片的函数
        # open_image(img_path)
        print(f"正在读取 {img_path}...")

        # 模拟：如果是 corrupt.jpg 就抛出错误
        if img_path == "corrupt.jpg":
            raise ValueError("文件头部损坏") # 人为制造一个错误

    except ValueError as e:
        # 只有发生 ValueError 时才会执行这里
        print(f"警告：跳过损坏的图片 {img_path}。错误信息：{e}")

    except FileNotFoundError:
        # 可以有多个 except 块来处理不同类型的错误
        print(f"警告：找不到文件 {img_path}")

    except Exception as e:
        # 捕获所有其他类型的错误（慎用！会隐藏 bug）
        print(f"发生了未知错误：{e}")

```

3. else 与 finally

else: 如果 try 块没有发生任何异常，执行这里。

finally: 无论是否发生异常，都会执行这里。通常用于清理资源（如关闭文件、断开数据库连接）。

```

try:
    f = open("data.txt", "r")

```

```

        content = f.read()

except FileNotFoundError:

    print("文件不存在")

else:

    print("读取成功，内容长度:", len(content))

finally:

    # 无论上面发生了什么，只要 f 被定义了，就尝试关闭它

    if 'f' in locals():

        f.close()

    print("清理工作完成。")

```

2.10.2 基础调试技巧 (logging / pdb / VS Code)

当代码运行结果不对时，许多新手的反应是满屏打 `print()`。虽然有用，但这很低效且不专业。

1. 使用 logging 模块替代 print

`print` 输出的信息混杂在一起，很难管理。Python 标准库的 `logging` 模块提供了分级的日志记录功能。

级别: `DEBUG < INFO < WARNING < ERROR < CRITICAL`

优势: 你可以随时通过配置，只显示 `ERROR` 级别的日志，而隐藏 `DEBUG` 日志，无需修改代码。

```

import logging

# 配置日志格式和级别

logging.basicConfig(

    level=logging.INFO, # 设置最低显示级别为 INFO

    format='%(asctime)s - %(levelname)s - %(message)s'

)

def calculate_loss(loss_val):

```

```
logging.debug(f'当前 loss 值: {loss_val}') # 调试信息（不会显示，因为级别  
低于 INFO）
```

```
if loss_val < 0:  
    logging.error("Loss 不能为负数！") # 错误信息（会显示）  
    return None
```

```
logging.info("计算完成") # 一般信息（会显示）  
return loss_val
```

```
calculate_loss(-0.5)
```

2. VS Code 图形化调试 (Visual Debugger)

这是最推荐的调试方式。我们在 3.2 节 简单提到过，这里再次强调其核心流程：

打断点 (Breakpoint)：在行号左侧点击红点。

F5 启动调试：程序停在断点处。

变量监视 (Watch)：在左侧窗口查看当前所有变量的值，甚至可以修改变量值来测试不同情况。

单步执行：一步步走代码，看逻辑分支是否符合预期。

3. pdb (Python Debugger)

这是 Python 内置的命令行调试器。虽然没有图形界面直观，但在服务器终端（没有图形界面）调试时非常有用。

用法：在代码中插入 `import pdb; pdb.set_trace()`，程序运行到这里会暂停，进入交互式调试模式。

常用命令：n (下一行), c (继续运行), p 变量名 (打印变量), q (退出)。

2.10.3 单元测试简介 (unittest / pytest)

写完代码怎么证明它是对的？靠“肉眼观察”或者“手动输入几个数试试”是不够的。我们需要编写测试代码来测试源代码。

单元测试 (Unit Testing) 是指对软件中的最小可测试单元（通常是函数或类）进行检查和验证。

1. assert 语句

这是最简单的测试手段。如果条件为真，无事发生；如果为假，抛出 `AssertionError`。

```
def add(x, y):  
    return x + y  
  
# 简单的断言测试  
assert add(1, 2) == 3, "1+2 应该等于 3"  
assert add(-1, 1) == 0, "-1+1 应该等于 0"  
print("所有断言通过！")
```

2. unittest vs. pytest

unittest: Python 标准库自带，模仿 Java 的 JUnit，需要写类，语法稍显繁琐。

pytest: 第三方库（需要 `pip install pytest`），AI 社区（如 PyTorch, TensorFlow）的首选。它语法极其简单，自动发现测试文件，输出报告清晰。

3. pytest 实战示例

假设我们有一个文件 `my_math.py`:

```
# my_math.py  
def divide(a, b):  
    if b == 0:  
        raise ValueError("除数不能为零")  
    return a / b
```

我们编写一个测试文件 `test_my_math.py`（文件名必须以 `test_` 开头）:

```
# test_my_math.py  
import pytest  
from my_math import divide  
  
# 测试正常情况  
def test_divide_normal():  
    assert divide(10, 2) == 5  
    assert divide(9, 3) == 3
```

```
# 测试异常情况：我们要确保它真的抛出了错误
```

```
def test_divide_zero():
```

```
    with pytest.raises(ValueError):
```

```
        divide(10, 0)
```

如何运行？

在终端输入 `pytest` 命令，它会自动找到所有 `test_*.py` 文件并运行。你会看到绿色的通过条（PASS）或红色的失败条（FAIL）。